

Estándar IEEE 16326 – ingeniería de sistemas y software – Proceso del ciclo de vida – Gestión de proyectos

Administración de proyectos 1

Integrantes

Ramírez	García	Oswaldo
Rios	Carrera	Jesús
Vargas	Angeles	Uriel
Osornio	Tapia	Jorge
Pérez Estrada	Guadalupe	Yosabeth

Matriculas

21-011-1167
19-011-0599
19-011-1318
21-011-0194
20-011-0820

Nombre del profesor
Máximo Eduardo Sánchez Gutiérrez

Historial de cambios

Versión	Fecha	Páginas afectadas	Descripción del cambio	Autor
1.0	02-03-2026	Todo el documento	Creación inicial del PMP	Jorge Tapia
1.1	17-03-2026	Secciones 5,6 y 7	Se agregó información del Sprint 2	Jorge Tapia
1.2	20-03-2026	Sección 8	Se integro sprint retrospective y gestión de riesgos	Jorge Tapia
1.3	22-03-2026	Sección 8.7	Se agregaron métricas (KPIs)	Jorge Tapia
1.4	24-04-2026	Sección 5.2 y 6	Se Integraron actividades WBS	Guadalupe Yosabeth Pérez Estrada. Jorge Edmundo Osornio Tapia
2.0	19-05-2026	Todo el documento	Modificación en todas las secciones	Guadalupe Yosabeth Pérez Estrada. Jorge Edmundo Osornio Tapia

Prefacio

El presente documento describe el Plan de Gestión del Proyecto (PMP) para el desarrollo de un sistema de comanda digital. Este documento está dirigido a todos los involucrados en el proyecto, incluyendo el equipo de desarrollo, stakeholders y responsables de la gestión del proyecto. Su propósito es establecer la organización, planificación, ejecución y control del proyecto, siguiendo buenas prácticas y estándares internacionales.

Contenido

Estándar IEEE 16326 – ingeniería de sistemas y software – Proceso del ciclo de vida –	
Gestión de proyectos	1
Administración de proyectos 1	1
Integrantes	1
Ramírez García OswaldoRios Carrera Jesús VicenteVargas Angeles UrielOsornio	
Tapia Jorge EdmundoPérez Estrada Guadalupe Yosabeth	1
Matriculas	1
21-011-116719-011-059919-011-1318	1
21-011-019420-011-0820	1
Nombre del profesorMáximo Eduardo Sánchez Gutiérrez	1
1. Visión general del proyecto	4
1.1 Resumen del proyecto	5
1.1.1 Propósito, alcance y objetivo	5
1.1.2 Supuestos y restricciones	7
1.1.3 Entregables del proyecto	11
Criterio de Cierre de Proyecto (Validación IEEE 16326)	14
1.1.4 Cronograma y presupuesto	14
Presupuesto y estimación de esfuerzo	15
Parámetros de estimación	16
Distribución del esfuerzo	16
1.2 Evolución del plan	18
A. Ciclo de Gestión de Cambios (Change Control)	18
B. Trazabilidad de la Evolución (Versiones del Pl	18
2. Definiciones	19
3. Referencias	19
4. Contexto del Proyecto	20
Antecedentes y justificación	20
Entorno operativo y tecnológico	20
Factores críticos de éxito	20
Restricciones del proyecto	20
4.1 Modelo de proceso	21

4.1.2 Justificación del modelo	22
4.2 Mejora del proceso	22
Mejora del Proceso	22
Objetivo de la Mejora Continua	22
Estrategia de Mejora Continua	22
Distinción de Niveles de Mejora	23
Mejoras Operativas del Proyecto	23
Mejoras Organizacionales	23
Métricas Utilizadas para la Mejora	24
Sprint Velocity	24
Burn-down Chart	24
Bug Rate	24
Cycle Time	24
Acciones de Mejora	24
Criterios de Éxito Medibles	25
Roles y Responsabilidades para la Mejora	25
Scrum Master	25
Product Owner	25
Equipo de Desarrollo	26
Responsable del Proyecto o Cliente	26
Gestión de Datos Históricos	26
Seguimiento de la Mejora	27
4.3 Plan de Infraestructura	27
Infraestructura de Trabajo	27
Infraestructura Tecnológica	28
Trello	28
GitHub	28
Docker	28
OneDrive	29
Red Local Wi-Fi del Restaurante	29
Herramientas de Ingeniería	29
Visual Studio Code	29
OpenCode	29

Composer y npm	29
MySQL	30
Sistemas Habilitadores	30
Control de Versiones	30
Contenerización	30
Comunicación en Red	30
Responsables de Infraestructura	31
Docker — Jesús Vicente Ríos Carrera	31
Trello — George Edmundo Osornio Tapia y Guadalupe Yosabeth Pérez Estrada	31
OpenCode — Oswaldo García Ramírez	31
GitHub — Todo el Equipo	31
Gestión y Mantenimiento de Infraestructura	31
4.4 Métodos, herramientas y técnicas	32
Metodologías de Desarrollo	32
Herramientas Utilizadas	32
Sistema Operativo — Fedora 44	32
XAMPP 8.0.30-0	33
Docker Engine 29.4.2	33
Docker Compose 5.1.2	33
PHP 8.2	33
MySQL 8.0	33
Node.js 22.22.2	34
npm 10.9.7	34
Playwright 1.59.1	34
PHPUnit 10.5	34
TestSprite MCP 0.0.37	34
Markdownify MCP 1.0.4	35
Git	35
Técnicas Utilizadas	35
Contenerización	35
Automatización de pruebas	35
Integración continua	35
Control de versiones	35

Pruebas End-to-End (E2E)	36
Pruebas unitarias	36
4.4.1 Herramientas del Gestion y seguimiento - Trello	36
Uso del Tablero Kanban	36
Definition of Done (DoD)	37
4.5 Plan de aceptación del producto	37
Criterios de Aceptación	37
Métodos de Aceptación	37
Plan de Pruebas de Aceptación	38
Acuerdo Formal de Aceptación	38
Evidencia de Aceptación	39
4.6 Organización del proyecto	39
Estructura Organizacional	39
Organización por Módulos Funcionales	39
Módulo de Autenticación y Control de Acceso	40
Asignación de Responsabilidades	40
4.6.1 Interfaces externas	41
Participación de Actores Externos	41
Dueño del Restaurante	41
Usuarios Finales	41
Comunicación y Retroalimentación	42
4.6.2 Interfaces internas	42
Comunicación Interna del Proyecto	42
Scrum Master	42
Product Owner	43
Desarrolladores Frontend y Backend	43
Mecanismos de Coordinación	44
4.6.3 Autoridades y Responsabilidades	44
5. Planificación del proyecto	45
5.1 Inicialización del proyecto	46
Inicio del Proyecto	46
Definición del Alcance	46
Identificación de Requerimientos	46

Selección de Metodología	47
Configuración del Entorno de Desarrollo	47
Organización del Equipo	47
Estimación Inicial	48
Validación Inicial	48
5.1.1 Plan de estimación	48
Objetivo del Plan de Estimación	48
Técnica de Estimación	49
Planning Poker	49
Unidad de Medición del Esfuerzo	49
Estimación de Tiempo del Proyecto	50
Estimación del Esfuerzo Total	50
Distribución del Esfuerzo	51
Métricas de Seguimiento y Estimación	51
Sprint Velocity	51
Burn-down Chart	51
Bug Rate	51
Cycle Time	51
Plan de Actualización de Estimaciones	52
Criterios de Éxito de la Estimación	52
Gestión de Datos Históricos	52
5.1.2 Plan de personal	53
Objetivo del Plan de Personal	53
Estructura del Equipo	53
Descripción de Roles	53
Scrum Master	54
Product Owner	54
Desarrolladores	54
Habilidades Requeridas	54
Estrategia de Adquisición de Personal	55
Plan de Formación Inicial	55
Estrategia de Colaboración y Soporte Técnico	56
Estrategia de Retención y Motivación	56

Seguimiento del Desempeño	56
Gestión de Conocimiento	57
5.1.3 Recursos	57
Objetivo del Plan de Recursos	57
Recursos de Hardware	57
Recursos de Software	58
Licencias de Software	58
Equipos de Prueba	59
Recursos Financieros	59
Recursos de Infraestructura	59
Sistemas Habilitadores	60
Gestión de Reservas y Contingencias	60
Seguimiento y Administración de Recursos	61
5.1.4 Entrenamiento	61
Objetivo del Entrenamiento	62
Alcance del Entrenamiento	62
Estrategia de Capacitación	62
Herramientas y Temas de Capacitación	63
Responsables del Entrenamiento	63
Evaluación del Entrenamiento	63
Actualización del Entrenamiento	63
5.2 Planes de trabajo	64
Objetivo de los Planes de Trabajo	64
Alcance del Trabajo	64
Implementación de Gestión de Mesas	64
Implementación del Menú Digital	65
Implementación del Módulo de Meseros	65
Implementación del Módulo de Cocina	65
Implementación del Módulo de Caja	66
Implementación del Panel de Administración	66
Organización del Trabajo por Sprint	67
Seguimiento del Trabajo	67
Validación del Trabajo	67

5.2.2 Actividades (WBS)	68
Objetivo de la WBS	68
Nivel 1 — Proyecto General	68
1. Sistema de Comanda Digital para Restaurante	68
Nivel 2 — Gestión y Planeación	68
1.1 Planeación del Proyecto	68
1.1.1 Definición de requerimientos	68
1.1.2 Organización del proyecto	68
1.1.3 Estimación y cronograma	69
1.1.4 Gestión documental	69
Nivel 2 — Configuración de Infraestructura	69
1.2 Configuración del Entorno	69
1.2.1 Configuración de Docker	69
1.2.2 Configuración del repositorio GitHub	69
1.2.3 Configuración de base de datos	69
1.2.4 Configuración de herramientas	69
Nivel 2 — Desarrollo del Sistema	69
1.3 Desarrollo de Funcionalidades	70
1.3.1 Implementación del módulo de autenticación	70
Actividades	70
1.3.2 Implementación de gestión de mesas	70
Actividades	70
1.3.3 Implementación del menú digital	70
Actividades	70
1.3.4 Implementación del módulo de meseros	70
Actividades	70
1.3.5 Implementación del módulo de cocina	70
Actividades	70
1.3.6 Implementación del módulo de caja	71
Actividades	71
1.3.7 Implementación del panel administrativo	71
Actividades	71
Nivel 2 — Integración y Pruebas	71

1.4 Validación del Sistema	71
1.4.1 Pruebas funcionales	71
1.4.2 Pruebas de integración	71
1.4.3 Pruebas operativas	71
1.4.4 Corrección de incidencias	71
Nivel 2 — Gestión del Proyecto Scrum	71
1.5 Seguimiento y Control	72
1.5.1 Daily Scrum	72
1.5.2 Sprint Planning	72
1.5.3 Sprint Review	72
1.5.4 Sprint Retrospective	72
1.5.5 Seguimiento de métricas	72
Nivel 2 — Despliegue y Cierre	72
1.6 Liberación del Sistema	72
1.6.1 Preparación de despliegue	72
1.6.2 Capacitación de usuarios	72
1.6.3 Aceptación final	72
1.6.4 Cierre del proyecto	73
5.2.3 Cronograma y presupuesto	73
Asignación del Cronograma	73
Objetivo del Cronograma	73
Estrategia de Planificación Temporal	73
Relaciones de Programación entre Actividades	74
Actividades Secuenciales	74
Actividades Paralelas	74
Restricciones del Cronograma	74
Cronograma General del Proyecto	75
Hitos del Proyecto	75
Hito 1 — Configuración Inicial del Proyecto	75
Sprint 1	75
Indicadores de cumplimiento	76
Hito 2 — Implementación Base del Sistema	76
Sprint 2	76

Indicadores de cumplimiento	76
Hito 3 — Integración Operativa del Restaurante	76
Sprint 3	76
Indicadores de cumplimiento	76
Hito 4 — Implementación Financiera y Administrativa	76
Sprint 4 y 5	76
Indicadores de cumplimiento	76
Hito 5 — Validación Final y Liberación	77
Sprint 6 y 7	77
Indicadores de cumplimiento	77
Técnicas de Representación del Cronograma	77
Asignación del Presupuesto	77
Objetivo del Presupuesto	77
Presupuesto por Categoría	77
Recursos Humanos	78
Infraestructura	78
Software y Herramientas	78
Reserva de Contingencia	78
Control Presupuestal	79
6. Evaluación y control	80
6.1 Gestión de requisitos	80
Objetivo de Evaluación y Control	80
Gestión de Requisitos	80
Obtención de Requisitos	81
Transformación de Requisitos a Historias de Usuario	81
Trazabilidad de Requisitos	81
Gestión de Cambios	82
Seguimiento y Control del Proyecto	82
Métricas de Evaluación	83
Sprint Velocity	83
Burn-down Chart	83
Bug Rate	83
Cycle Time	83

Evaluación de Calidad	83
Gestión de Incidencias	84
Control del Cronograma y Presupuesto	84
Gestión de Evidencias y Reportes	84
Mejora Continua	85
6.2 Control de alcance Objetivo del Control de Alcance	85
Línea Base del Alcance	85
Proceso de Control de Cambios	86
1. Solicitud del cambio	86
2. Análisis técnico y operativo	86
3. Evaluación de impacto	86
4. Revisión y aprobación	86
5. Actualización documental	87
Comité de Control de Cambios	87
Gestión de Prioridades	87
Control de Scope Creep	88
Seguimiento del Alcance	88
Evidencia y Trazabilidad	88
6.3 Control de cronograma	89
Objetivo del Control del Cronograma	89
Línea Base del Cronograma	89
Mecanismos de Seguimiento del Cronograma	89
Seguimiento de Hitos	90
Métricas de Control del Cronograma	90
Sprint Velocity	90
Burn-down Chart	90
Cycle Time	91
Técnica de Valor Ganado (Earned Value Management)	91
Planned Value (PV)	91
Earned Value (EV)	91
Schedule Variance (SV)	91
Schedule Performance Index (SPI)	91
Acciones Correctivas del Cronograma	92

Frecuencia de Evaluación	92
Gestión de Evidencias del Cronograma	93
Control del Presupuesto	93
Objetivo del Control Presupuestal	93
Línea Base del Presupuesto	93
Seguimiento del Presupuesto	94
Métricas de Control Presupuestal	94
Actual Cost (AC)	94
Cost Variance (CV)	94
Cost Performance Index (CPI)	94
Informes de Control Presupuestal	95
Acciones Correctivas Presupuestales	95
6.4 Control de presupuesto	95
Objetivo del Control Presupuestal	96
Línea Base del Presupuesto	96
Métricas de Costo y Esfuerzo	96
Técnica de Valor Ganado	96
Actual Cost (AC)	96
Earned Value (EV)	97
Cost Variance (CV)	97
Cost Performance Index (CPI)	97
Aprobación de Gastos	97
Gestión de Reservas de Contingencia	98
Frecuencia de Evaluación Presupuestal	98
Acciones Correctivas	98
Evidencia y Reportes	99
6.5 Aseguramiento de la calidad	99
Objetivo del Aseguramiento de la Calidad	99
Alcance del Aseguramiento de Calidad	100
Auditorías y Revisiones Internas	100
Validación de Requisitos y Funcionalidades	100
Revisiones Técnicas	101
Estrategia de Pruebas	101

Pruebas Funcionales	101
Pruebas de Integración	101
Pruebas de Caja Negra	102
Pruebas Operativas	102
Estándares Aplicados	102
Mejora Continua (PDCA)	102
Plan (Planificar)	102
Do (Ejecutar)	102
Check (Verificar)	103
Act (Actuar)	103
Métricas de Calidad	103
Densidad de Defectos	103
Número de Incidencias Críticas por Sprint	103
Sprint Velocity	103
Resultados de Pruebas Funcionales e Integración	103
Bug Rate	104
Gestión de Incidencias de Calidad	104
Evidencia y Trazabilidad de Calidad	104
Responsabilidades de Calidad	104
6.6 Subcontratistas	105
Objetivo	105
Situación Actual del Proyecto	105
Posibles Servicios Externos Futuros	105
Criterios de Selección de Subcontratistas	106
Supervisión y Control	106
Riesgos Relacionados con Subcontratación	106
Evidencia y Contratos	107
6.7 Cierre del proyecto	107
Objetivo del Cierre del Proyecto	107
Criterios de Cierre	107
Actividades de Cierre	108
Validación Final del Sistema	108
Consolidación de Evidencias	108

Respaldo y Liberación Final	109
Acta de Cierre	109
Informe Final del Proyecto	109
Análisis de Desempeño vs Planeación	110
Lecciones Aprendidas	110
Cierre Administrativo	111
Responsables del Cierre	111
7. Entrega del producto	111
Objetivo de la Entrega del Proyecto	111
Transferencia Técnica y Operativa	112
Estrategia de Entrega	112
Entregables Finales	113
Documentación del Proyecto	113
Capacitación	114
Soporte Post-Entrega	114
Periodo de Garantía	115
Validación de la Entrega	115
Responsables de la Entrega	115
Planes de procesos de soporte	116
8.1 Supervisión y Entorno de Trabajo	116
Objetivo	116
Supervisión del Progreso	116
Ambiente de Trabajo	116
Seguridad y Salud Ocupacional	117
Bienestar del Equipo	117
Mecanismos de Escalamiento	117
Problemas Técnicos del Sistema	117
Problemas de Infraestructura o Equipo	118
Problemas de Tiempo	118
8.2 Gestión de Decisiones	118
Objetivo	118
Categorías de Decisión	118
Estrategia de Decisión	119

Registro de Decisiones	119
Riesgos por Sprint	119
8.3 Gestión de Riesgos	120
Objetivo	120
Estrategia de Identificación	120
Análisis y Priorización	120
Seguimiento	120
Matriz de Riesgos	120
8.4 Gestión de Configuración	121
Control de Versiones	121
Gestión de Cambios	122
Estrategia de Versionamiento	122
8.5 Gestión de la Información	122
Gestión Documental	122
Seguridad de la Información	122
Comunicación del Proyecto	122
Mecanismo de Transferencia de Información	123
Matriz de Comunicación	123
8.6 Aseguramiento de Calidad (QA)	124
Revisiones	124
Estándares Aplicados	124
8.7 Medición	124
Objetivo	124
Métricas Utilizadas	124
Sprint Velocity	124
Burn-down Chart	125
Bug Rate	125
Cycle Time	125
8.8 Revisiones y Auditorías	125
Revisiones Internas	125
Revisiones con Cliente	125
8.9 Verificación y Validación	125
Pruebas Unitarias	125

Pruebas de Integración	125
Pruebas de Estrés	125
Criterios de Aceptación	126
8. Planes adicionales	126
Objetivo	126
8.1 Plan de Continuidad Operativa	126
Objetivo	126
Estrategias de Continuidad	126
Recuperación ante Fallos	126
8.2 Plan de Mantenimiento	127
Objetivo	127
Actividades de Mantenimiento	127
8.3 Plan de Seguridad	127
Objetivo	127
Medidas de Seguridad	127
Seguridad Operativa	127
8.4 Plan de Respaldo y Recuperación	128
Objetivo	128
Estrategia de Respaldo	128
Frecuencia de Respaldo	128
Ubicación de Respaldos	128
8.5 Plan de Mejora Continua	128
Objetivo	128
Estrategia de Mejora	128
Acciones de Mejora	129
Criterios de Éxito	129
8.6 Plan de Soporte Post-Entrega	129
Objetivo	129
Alcance del Soporte	129
Periodo de Soporte	129
8.7 Plan de Gestión del Conocimiento	130
Objetivo	130
Estrategias	130

Repositorio de Información	130
8.8 Plan de Gestión de Lecciones Aprendidas	130
Objetivo	130
Registro de Lecciones Aprendidas	130
Información Registrada	131
Responsables	131
Anexos	131
Anexo A – Product Backlog	131
Anexo B – Sprint Planning	131
Anexo C – Sprint Review	131
Anexo E – Sprint Retrospective	132
Anexo F – Daily Scrum	132
Anexo F –KPIs	132
A.1 Sprint Velocity	132
A.2 Burn-down Chart	133
A.3 Bug Rate	134
A.4 Cycle Time	134
A.5 Lead Time	136
A.6 Troughput	136
Anexo G – Kanban	137
Anexo H – Cronograma de los sprints	137
Anexo I – Presupuesto	137
Anexo J – Plan de entrenamiento	137
Anexo K – Gestión de Configuración e integración de cambios Git	137
Anexo L – Pruebas	137

1. Visión general del proyecto

1.1 Resumen del proyecto

El **Sistema de Comanda Digital** surge como una solución tecnológica integral para la modernización de la industria restaurantera. El proyecto no solo busca digitalizar el papel, sino optimizar el ecosistema de comunicación entre tres pilares fundamentales: **Cliente (Comensal)**, **Producción (Cocina/Barra)** y **Administración (Caja/Gerencia)**. Mediante una arquitectura cliente-servidor, el sistema permite la toma de pedidos de forma autónoma por parte del cliente a través de dispositivos móviles o tablets situadas en las mesas. Esta información viaja en tiempo real hacia una interfaz de gestión en cocina, eliminando los "cuellos de botella" informativos, reduciendo el margen de error humano en un 90% y agilizando el tiempo de respuesta del servicio.

Problemas

La dependencia de comandas físicas genera lentitud en horas pico, errores en la preparación por letra ilegible y una desconexión entre el inventario real y las ventas registradas.

Solución

Una plataforma desarrollada en **PHP y SQL** que permite la toma de pedidos mediante dispositivos móviles, con actualización automática en pantallas de cocina y un dashboard administrativo para el control de inventarios y reportes financieros.

1.1.1 Propósito, alcance y objetivo

La norma IEEE 16326 exige definir claramente qué se va a hacer (propósito), hasta dónde llega el proyecto (alcance) y qué se espera lograr (objetivos) para evitar la expansión descontrolada del alcance (*creeping scope*). En perfecta concordancia con el estándar de descripción arquitectónica **ISO/IEC/IEEE 42010:2011**, esta delimitación formaliza la estructura base del sistema y establece un lenguaje común entre todos los *stakeholders* (desarrolladores, personal operativo del restaurante y administradores), asegurando que la solución evolucione de forma ordenada, escalable y sin desviarse de las necesidades prioritarias del negocio.

PROPÓSITO

El propósito de este proyecto es desarrollar e implementar la arquitectura del **Sistema de Comanda Digital (SCD)**, un sistema web orientado a la digitalización y optimización del proceso de toma y gestión de pedidos dentro del restaurante. El sistema busca sustituir de manera definitiva las comandas físicas, reducir

drásticamente los errores de comunicación y optimizar los tiempos de servicio mediante una guía técnica estructurada que abarque desde la interacción del cliente en la mesa hasta la coordinación en cocina y la validación en caja.

ALCANCE

El alcance de este proyecto se centra exclusivamente en el diseño y desarrollo arquitectónico de la aplicación web responsiva adaptada a las operaciones internas del restaurante.

Dentro del alcance:

- **Toma de pedidos en mesa:** Interfaz web responsiva diseñada para ejecutarse en *tablets* instaladas en cada mesa, incorporando una identificación única por dispositivo (Id_Mesa).
- **Personalización de pedidos:** Módulo interactivo que permite al cliente modificar sus platillos en tiempo real (agregar o quitar ingredientes, especificar cantidades y añadir comentarios especiales).
- **Comunicación en tiempo real:** Flujo síncrono de envío de datos desde las *tablets* de las mesas hacia una pantalla de visualización centralizada en el área de cocina.
- **Módulo de caja y pre-facturación:** Generación e impresión de tickets físicos para la validación del pedido y cobro en caja previo a la preparación en cocina.
- **Función de asistencia digital:** Herramienta integrada en la interfaz de la *tablet* para que el cliente solicite el apoyo presencial de un mesero de forma inmediata.
- **Gestión administrativa básica:** Panel de control para el administrador orientado al seguimiento básico del inventario de insumos y generación de reportes operativos.
- **Aseguramiento de calidad y cierre:** Ejecución de pruebas unitarias, de integración y de aceptación por cada componente de la arquitectura, acompañadas de su respectiva documentación técnica y manuales de usuario.

Fuera del alcance (Lo que NO se incluye):

- **Procesamiento de pagos en línea:** No se contempla la integración con pasarelas de pago externas (Stripe, PayPal) ni sistemas bancarios; el registro de cobro se gestionará de manera física/manual en caja tras la impresión del ticket.
- **Análítica empresarial avanzada:** Exclusión de funciones complejas de *Business Intelligence* (BI) o integraciones con sistemas externos de gestión empresarial (como ERPs o CRMs comerciales).

- **Desarrollo móvil nativo:** El sistema no contará con aplicaciones instalables para iOS o Android; su ejecución se limitará estrictamente a navegadores web modernos para garantizar compatibilidad multiplataforma.
- **Infraestructura en la nube compleja:** El despliegue inicial está diseñado para un entorno de servidor local (Apache/MySQL), por lo que se excluyen arquitecturas auto-escalables en la nube (AWS, Azure).

OBJETIVOS

Los objetivos del proyecto se definen bajo el criterio SMART (Específicos, Medibles, Alcanzables, Relevantes y Temporales):

1. **Eficiencia en el servicio:** Reducir en un 30% los tiempos de espera del cliente en comparación con el método tradicional de comandas de papel, medido desde el envío en la *tablet* hasta la entrega del platillo.
2. **Mitigación de errores operativos:** Reducir en un 90% los errores de transcripción y comunicación interna en la cocina provocados por letra ilegible o malentendidos verbales.
3. **Trazabilidad total de consumos:** Garantizar el 100% de la correlación de los pedidos con su mesa de origen mediante la correcta vinculación del identificador único de dispositivo (Id_Mesa).
4. **Estabilidad del Software:** Mantener una tasa de error (*Bug Rate*) inferior al 20% durante las pruebas de aceptación técnica al cierre de cada ciclo de desarrollo.
5. **Cumplimiento del Cronograma:** Completar y entregar el sistema web completamente funcional en un periodo cerrado de **8 sprints (16 semanas)**, concluyendo de manera definitiva en junio de 2026.
6. **Rendimiento del equipo:** Mantener una velocidad de desarrollo promedio constante de entre 25 y 30 *Story Points* por sprint.

1.1.2 Supuestos y restricciones

Restricciones

Las restricciones representan limitaciones reales, técnicas, organizacionales o legales impuestas al proyecto que el equipo de desarrollo no puede modificar. Su incumplimiento compromete directamente la viabilidad del sistema.

Categoría	Descripción de la restricción	Consecuencia de incumplimiento
Tecnológica	El sistema debe ejecutarse obligatoriamente sobre el	Incompatibilidad con el entorno de despliegue final en el restaurante y

	<i>stack</i> local compuesto por PHP 8.2, MySQL 8.0 y Apache 2.4. No se permite el uso de otros lenguajes de backend o motores de bases de datos.	rechazo técnico de la entrega.
Temporal	El proyecto cuenta con un plazo estricto e inamovible de 8 sprints (16 semanas), iniciando el 02 de marzo de 2026 y finalizando el 28 de mayo de 2026.	Reprobación académica del proyecto de certificación y penalización por incumplimiento de cronograma ante los <i>stakeholders</i> .
Presupuestaria	El presupuesto financiero asignado al proyecto es de \$0.00 MXN. Se restringe el desarrollo al uso exclusivo de tecnologías de código abierto, <i>software</i> libre y hardware preexistente.	Detención del desarrollo por incapacidad de adquirir licencias comerciales o servicios de pago.
Seguridad y Privacidad	Es obligatorio implementar un control de acceso basado en roles (RBAC) estricto. Toda la información de comandas y ventas debe almacenarse localmente, prohibiendo explícitamente el uso de servicios de almacenamiento en la nube.	Vulnerabilidad ante accesos no autorizados a funciones críticas.
Personal	El equipo está limitado estrictamente a 3 desarrolladores con roles polivalentes, disponiendo de un máximo de 5 horas diarias por integrante para el proyecto.	Reducción de la velocidad del sprint (<i>Sprint Velocity</i>), sobrecarga de trabajo y omisión forzada de entregables planificados.
Operativa / Usabilidad	El diseño de la interfaz web debe ser lo suficientemente intuitivo para que el personal de	Resistencia al uso por parte del personal del restaurante, ralentización del servicio y aumento de

		cocina y meseros no requiera más de 30 minutos de capacitación total para operar el sistema con fluidez.	errores humanos en la gestión de comandas.
Cumplimiento de Estándares		El diseño arquitectónico está obligado a seguir estrictamente la estructura de vistas y puntos de vista dictados por el estándar ISO/IEC/IEEE 42010:2011 .	El documento perderá su validez de certificación y consistencia formal ante los evaluadores del proyecto.
Compatibilidad de Hardware (Clientes)		Las <i>tablets</i> de las mesas y la pantalla de cocina deben contar con navegadores modernos que soporten de forma nativa HTML5, CSS3 y JavaScript moderno (ES6+) .	Fallos en la renderización de la interfaz, pérdida de responsividad y mal funcionamiento de los scripts en tiempo real.
Dependencia del Entorno		El sistema debe ser capaz de operar al 100% de su capacidad sin requerir conexión a internet externa , dependiendo únicamente de la red de área local (LAN).	Caída total del sistema si el backend intenta consumir dependencias, tipografías o librerías mediante CDNs externas en lugar de locales.
Legal / Normativa		El sistema debe cumplir con la normativa local de protección de datos (ej. LFPDPPP en México) si se almacena información de empleados, y no debe registrar datos fiscales regulatorios complejos al estar fuera del alcance de facturación.	Sanciones legales por mal manejo de datos de usuarios o malentendidos sobre la validez fiscal de los tickets impresos.

Supuestos

Los supuestos son factores o premisas que el equipo de desarrollo da por verdaderos para fines de planificación, pero que están fuera de su control directo. Si

un supuesto resulta ser falso, se transforma inmediatamente en un riesgo para el proyecto.

Supuesto de Proyecto	Impacto si resulta falso	Riesgo asociado en §8.3
Infraestructura de Red: El restaurante dispondrá de una red Wi-Fi local dedicada, estable y con cobertura total tanto en el área de mesas como en la cocina central.	Pérdida de la sincronización en tiempo real de los pedidos; las <i>tablets</i> no podrán enviar las comandas a la pantalla de cocina ni a la caja.	R-01: Interrupción del servicio por fallo en la infraestructura de red local del cliente.
Hardware de Servidor: El cliente proveerá un equipo de cómputo que funcionará como servidor local con las especificaciones mínimas de hardware necesarias para ejecutar Apache y MySQL de forma fluida.	Degradación severa del rendimiento del sistema, latencia alta en la respuesta de las <i>tablets</i> y caídas constantes del servicio bajo alta demanda.	R-02: Incompatibilidad o insuficiencia del hardware del servidor local del restaurante.
Disponibilidad del Equipo: Todos los integrantes del equipo mantendrán una disponibilidad constante y un estado de salud óptimo durante las 16 semanas de ejecución del proyecto.	Aparición de cuellos de botella en módulos críticos, sobrecarga de tareas en los miembros restantes y retraso en el cierre de los sprints.	R-03: Ausentismo o baja imprevista de un miembro del equipo de desarrollo.
Hardware de Pruebas: Se dispondrá de al menos una <i>tablet</i> física funcional desde el inicio de las etapas de pruebas para validar la responsividad y la asignación del Id_Mesa.	Imposibilidad de validar la usabilidad real del sistema en producción, generando fallos ocultos de visualización o interacción en los dispositivos finales.	R-04: Carencia de dispositivos físicos para pruebas de aceptación en entorno real.
Entrega de Información (Menú): El administrador del restaurante proporcionará la lista completa de productos, precios, categorías e insumos a más tardar al finalizar el Sprint 2.	Retraso en la carga de la base de datos de prueba, impidiendo validar el comportamiento real del menú digital en los sprints intermedios.	R-05: Retraso en entregables por falta de insumos de información por parte del cliente.

Suministro Eléctrico Estable: El restaurante cuenta con una instalación eléctrica regulada o un sistema de energía ininterrumpida (UPS) para el servidor y la pantalla de cocina.	Un apagón corromperá la base de datos de MySQL en pleno servicio, provocando pérdida de comandas activas y paro total del restaurante.	R-06: Pérdida de integridad de datos por fallos en el suministro eléctrico local.
Disposición al Cambio (Cultura Organizacional): El personal operativo (meseros y cocineros) mostrará una actitud cooperativa durante las pruebas piloto y la fase de levantamiento de requerimientos.	Boicot pasivo del sistema, datos de retroalimentación sesgados y reportes de "fallos" que en realidad son resistencia al uso de la tecnología.	R-07: Resistencia al cambio y baja adopción tecnológica por el personal operativo.

1.1.3 Entregables del proyecto

Para garantizar la transparencia y la correcta gestión de la configuración, los entregables se clasifican en tres categorías obligatorias: Entregables de Software (Producto), Entregables Técnicos (Ingeniería y Arquitectura) y Entregables de Gestión y Calidad.

A. Entregables de Producto

Representa el software ejecutable y los esquemas de datos listos para su despliegue definitivo en el entorno del restaurante.

ID	Entregable / Documento	Componentes Reales Incluidos	Formato de Entrega	Estado
ENT-PROD-01	Base de Datos Relacional (SCD)	Esquema centralizado en MySQL. Incluye tablas optimizadas de usuarios, roles, catálogo de menú, persistencia de comandas e Id_Mesa.	Script .sql estructurado con diccionario de datos.	COMPLETADO (Listo para producción)
ENT-PROD-02	Sistema de Comanda	Aplicación web responsiva	Código fuente	En proceso

	Digital (Código Fuente)	completa (Sprints 2 al 7). Incluye lógica backend en PHP (PDO), patrón Observer, interfaces de usuario (Cliente, Cocina, Mesero, Caja, Administrador), suites de seguridad, <i>rate limiting</i> y empaquetado final.	completo en contenedor Dockerfile.	
--	-------------------------	---	---	--

B. Entregables de Ingeniería y Gestión de Calidad (Documentación)

Documentos formales y planes de ingeniería que validan el diseño, la mitigación de fallos y garantizan la transferencia tecnológica al cliente.

ID	Entregable / Documento	Componentes Reales Incluidos	Formato de Entrega	Estado Actual
ENT-ING-01	Plan de Gestión de Riesgos	Matriz formal de riesgos (§8.3) con análisis de impacto, probabilidad y estrategias de mitigación implementadas.	Documento digital PDF.	En proceso
ENT-ING-02	Descripción de Arquitectura (AD)	Documentación del diseño arquitectónico del SCD bajo las vistas del estándar ISO/IEC/IEEE 42010:2011.	Documento digital PDF.	COMPLETADO
ENT-ING-03	Plan de Pruebas	Estrategia y resultados de las pruebas unitarias, de	Documento digital PDF.	En proceso

		integración, seguridad y rendimiento con un <i>Bug Rate</i> verificado < 20%.		
ENT-ING-04	Manual Técnico del Sistema	Especificación del diseño lógico (IEEE 1016), diagramas <i>Nassi-Shneiderman</i> , diagramas de clases y documentación de la arquitectura PDO/Observer.	Documento digital PDF.	En proceso
ENT-ING-05	Manual de Usuario y Operación	Guía visual e interactiva paso a paso para los roles del restaurante (Administrador, Mesero, Cocina, Caja).	Documento digital PDF.	En proceso

C. Entregables de Gobernanza Ágil (Histórico de Ciclo de Vida)

Artefactos metodológicos que evidencian la gestión, el ritmo de desarrollo y la transparencia del equipo durante las 16 semanas.

ID	Entregable / Documento	Propósito de Control (IEEE 16326)	Formato / Registro	Estado Actual
ENT-AGI-01	Product Backlog	Listado maestro de historias de usuario que gobernó el alcance del sistema.	Documento PDF.	COMPLETADO

ENT-AGI-02	Sprint Planning (S1 - S8)	Planes de alcance quincenales y compromisos de <i>Story Points</i> asignados.	Minutas de planeación por Sprint.	En proceso
ENT-AGI-03	Daily Scrum (Bitácora)	Registro diario de sincronización del equipo y resolución de impedimentos.	Bitácora técnica de seguimiento.	En proceso
ENT-AGI-04	Sprint Review & Retrospective	Evaluaciones de incrementos de software y lecciones aprendidas de los procesos de desarrollo.	Informes ejecutivos post-sprint.	En proceso

Criterio de Cierre de Proyecto (Validación IEEE 16326)

Para que el proyecto se declare formalmente en estado **Cerrado y Aceptado**, se deben cumplir las siguientes condiciones de esta sección:

1. El sistema de comanda (ENT-PROD-02) debe cambiar su estatus a completado al finalizar todas sus pruebas e implementaciones.
2. El plan de **Plan de Gestión de Riesgos (ENT-ING-01)** debe cambiar su estatus a **completado tras su última actualización**.
3. **El Plan de Pruebas**
4. **El Product Backlog, Sprint Planning (S1 - S8), Daily Scrum (Bitácora) y Sprint Review & Retrospective deben cambiar su estatus a completado tras su última actualización**
5. El **Manual de Usuario y Operación (ENT-ING-05)** debe cambiar su estatus a *Completado* tras una última prueba de legibilidad con el personal del restaurante.
6. Se debe realizar el despliegue de la **Base de Datos (ENT-PROD-01)** y el **Código Fuente (ENT-PROD-02)** en el servidor local del restaurante utilizando las instrucciones descritas en el Manual Técnico.
7. Se firmará el **Acta de Cierre Final** por parte de los integrantes del equipo y los evaluadores/stakeholders, certificando que el alcance se cumplió sin variaciones descontroladas (*creeping scope*).

1.1.4 Cronograma y presupuesto

El proyecto se rige por un cronograma de 7 Sprints (14 semanas), donde el presupuesto se define en términos de **esfuerzo humano (horas-hombre)**, dado que el desarrollo es de fuente abierta y recursos propios.

A continuación, la **Tabla** detalla la relación entre los hitos temporales y la carga de trabajo estimada para el equipo:

Sprint	Hito / Fases	Esfuerzo (HRS)	Entregables principales	Estado
Sprint 1	Iniciación y Arquitectura	40	Backlog.	Completado
Sprint 2	Núcleo y Mesas	40	Módulo de Mesas, Validación, Login, Plan de Pruebas.	Completado
Sprint 3	Flujo de Pedidos	36	UI Cocina, Carrito, Lógica de Pedidos (FIFO).	Completado
Sprint 4	Operación Caja	40	UI Caja, Generación de Tickets, Cálculo de Cambio.	Completado
Sprint 5	Administración	40	Dashboard, CRUD Productos/Inventario, Observer.	Completado
Sprint 6	Calidad y Seguridad	40	Pruebas (Unitarias/Seguridad), Rate Limiting.	Completado
Sprint 7	Integración y Despliegue. Cierre y Aceptación	40	Docker, Pipeline CI/CD, Manuales, Pruebas Rendimiento. Manual de Usuario, Acta de Cierre, Liberación Final.	Proceso

Nota de referencia: El diagrama de Gantt, la distribución de tareas y el registro histórico del flujo de trabajo (capturas de tablero Kanban) se encuentran documentados en el archivo adjunto: **"Anexo B: Cronograma de Sprints y Evidencias de Gestión Ágil"**. Este documento es parte integral de la documentación del proyecto y debe consultarse para el control de desviaciones temporales.

Presupuesto y estimación de esfuerzo

Dado que el Sistema de Comanda Digital (SCD) es un proyecto de desarrollo basado en *software* libre y hardware preexistente, el presupuesto se define mediante el **Costo de Esfuerzo en Horas-Hombre**. Esta métrica es el indicador estándar para medir la inversión real de tiempo requerida para completar los entregables identificados en el inventario del proyecto.

Parámetros de estimación

Para el cálculo del esfuerzo se han definido los siguientes parámetros operativos:

- Equipo de trabajo: 3 integrantes
- Capacidad individual: 4 horas efectivas de desarrollo
- Capacidad del equipo: 12 horas-hombre por día
- Periodo de ejecución: 14 semanas (70 días hábiles de desarrollo activo).
- Esfuerzo estimado: 7
- $3(\text{personas}) \times 4(\text{horas}) \times 70(\text{días}) = 840$ horas hombre totales

Distribución del esfuerzo

La distribución del presupuesto de tiempo se ha categorizado según las fases del ciclo de vida del software, permitiendo una trazabilidad clara entre el diseño, la construcción y el aseguramiento de la calidad.

Tabla: Presupuesto de Esfuerzo por Categoría (IEEE 16326)

El esfuerzo total del proyecto fue calculado considerando el número de integrantes del equipo, las horas efectivas de trabajo por día y el periodo total de desarrollo activo. La ecuación utilizada para la estimación fue la siguiente:

$$E=P \times H \times D$$

Donde:

- E = Esfuerzo total estimado en horas-hombre
- P = Número de personas del equipo
- H = Horas efectivas de trabajo por día
- D = Días hábiles de desarrollo

Sustituyendo los valores del proyecto:

$$E=3 \times 4 \times 70$$

Resultado:

$$E=840 \text{ horas-hombre}$$

Los porcentajes asignados a cada categoría del proyecto fueron definidos considerando la complejidad, tiempo requerido y nivel de participación de cada fase dentro del ciclo de vida del software, tomando como referencias prácticas de estimación utilizadas en proyectos de desarrollo bajo IEEE 16326.

La fase de **Construcción (40%)** recibió el mayor porcentaje debido a que concentra las actividades de programación, integración de módulos, desarrollo de frontend y backend, siendo la etapa con mayor carga técnica y operativa.

$$840 \times 0.40 = 336$$

La categoría de **Gestión y Documentación (25%)** contempla actividades administrativas y de seguimiento como planeación, backlog, gestión de riesgos, elaboración de manuales y documentación técnica, las cuales son necesarias durante casi todo el proyecto.

$$840 \times 0.25 = 210$$

La fase de **Arquitectura y Diseño (15%)** incluye el modelado de la base de datos, diseño de arquitectura y documentación IEEE 1016. Aunque es una etapa crítica, su duración es menor comparada con la construcción.

$$840 \times 0.15 = 126$$

El área de **Pruebas y QA (15%)** considera pruebas unitarias, integración y validación funcional para asegurar la calidad del software y reducir errores antes de la entrega final.

$$840 \times 0.15 = 126$$

Finalmente, **Despliegue y Cierre (5%)** corresponde a actividades finales como configuración del entorno, despliegue con Docker, validación de aceptación y entrega del producto.

$$840 \times 0.05 = 42$$

Categoría de Esfuerzo	Horas Totales	% de Esfuerzo	Descripción de Actividades
Gestión y Documentación	210	25%	Planeación, Backlog, Manuales, Riesgos, Actas.
Arquitectura y Diseño	126	15%	SDD (IEEE 1016), Diseño BD.
Construcción (Coding)	336	40%	Desarrollo backend, frontend, lógica y módulos.

Pruebas y QA (Calidad)	126	15%	Pruebas unitarias, de seguridad e integración.
Despliegue y Cierre	42	5%	Docker, Aceptación final y entrega.
TOTAL	840	100%	Esfuerzo Total Requerido

1.2 Evolución del plan

De acuerdo con el estándar IEEE 16326, la planificación de un proyecto de software es un proceso dinámico que debe adaptarse a los hallazgos técnicos durante la ejecución. La evolución de nuestro plan se gestionó mediante dos mecanismos de control:

A. Ciclo de Gestión de Cambios (Change Control)

Ante cualquier desviación identificada en el seguimiento semanal (Sprint Review), se aplicó el siguiente protocolo:

1. **Identificación:** Detección de la desviación en el cronograma o esfuerzo.
2. **Impact Analysis:** Evaluación del impacto sobre el presupuesto total (las 1,280 horas-hombre).
3. **Aprobación:** Revisión por parte del equipo para ajustar las tareas del siguiente Sprint.
4. **Registro:** Actualización de los artefactos de gestión (Cronograma y Tableros Kanban)

B. Trazabilidad de la Evolución (Versiones del Pl

El plan de proyecto no fue una "foto fija", sino un documento con control de versiones. La evolución se resume en la siguiente tabla de hitos de actualización:

Versión	Fecha	Motivo de la Actualización	Impacto en Horas/Esfuerzo
V1.0	02-03-2026	Planificación inicial	Ninguno
V1.1	17-03-2026	Ajuste de tiempos de Sprint	Reasignación de horas

V1.2	20-03-2026	Integración de información de sprint retrospective y gestion de riesgos	Incremento de 2 horas de documentación
V1.3	22-03-2026	Integración de metricas KPIs	Incremento de 2 horas de análisis y seguimiento
V1.4	24-04-2026	Integración de actividades WBS	Ajuste organizacional sin impacto

2. Definiciones

En la siguiente tabla se encuentra la definición de conceptos clave del dominio del sistema

Termino	Definición
Backlog	Artefacto central de gestión ágil de alcance
Bug Rate	Métrica de calidad del proyecto
Burn-down Chart	Herramienta de seguimiento de avance
Ciclo DevOps	Fases del ciclo de vida del proyecto
Criterios de aceptación	Validación de producto contra requisitos
Cronograma de Sprints	Línea base del cronograma
Cycle Time	Métrica de eficiencia del equipo
Definition of Done (DoD)	Estándar de calidad para entregables
Entregable	Producto tangible de cada sprint
Historia de usuario	Unidad funcional de alcance
IEEE 16326	Estándar que rige el PMP
KPI	Indicadores de rendimiento del proyecto
Lead Time	Métrica de tiempo total de entrega
Plan de gestión de proyecto (PMP)	El mismo documento
Prioridad	Criterio de ordenamiento del backlog
Product Owner	Rol responsable del valor del producto
Riesgo	Eventos potenciales con plan de mitigación
Rol de usuario	Perfiles de acceso del sistema
Scrum	Metodología de gestión del proyecto
Scrum Máster	Rol que facilita el proceso
Sprint	Unidad de tiempo del cronograma
Sprint Velocity	Métricas de capacidad del equipo

Story point	Unidad de estimación de esfuerzo
Throughput	Métrica de productividad
UAT	Validación con el cliente
Versionado semántico	Gestión de releases y entregas

3. Referencias

1. IEEE, *IEEE Standard for Project Management in IT/Software Projects*, IEEE Std 16326-2009, 2009.
2. ISO/IEEE, *Systems and software engineering — Architecture description*, ISO/IEC/IEEE 42010:2011, 2011.
3. Documento de Diseño de Arquitectura (TDD), Backlog de Producto y Guía de Estilos de Interfaz

4. Contexto del Proyecto

Antecedentes y justificación

El Sistema de Comanda Digital (SCD) surge como respuesta a la necesidad de optimizar la comunicación operativa entre el área de atención al cliente y el área de cocina en establecimientos gastronómicos de mediana escala. Los métodos tradicionales (papel) presentan cuellos de botella en la gestión de pedidos, errores en la captura de órdenes y una falta de trazabilidad en los tiempos de despacho.

Entorno operativo y tecnológico

El proyecto se desarrolla bajo un ecosistema de **software libre**, aprovechando tecnologías de código abierto para asegurar la escalabilidad y reducir costos de licenciamiento. El entorno operativo objetivo es un modelo cliente-servidor con despliegue en contenedores (*Docker*), garantizando la portabilidad entre diferentes infraestructuras de red local.

Factores críticos de éxito

Para que el SCD sea considerado un proyecto exitoso, se han definido los siguientes factores:

- **Integridad de Datos:** Garantizar que no existan inconsistencias entre el pedido realizado por el mesero y el recibido en cocina.
- **Latencia de Comunicación:** Tiempo de respuesta del sistema inferior a 2 segundos bajo carga concurrente.
- **Usabilidad:** Curva de aprendizaje para el personal operativo inferior a 2 horas.

- **Mantenibilidad:** Código estructurado bajo estándares de diseño (Patrones de diseño) que permitan futuras actualizaciones por terceros.

Restricciones del proyecto

- **Recursos:** El desarrollo se realiza con recursos de equipo autogestionados (sin presupuesto de capital para infraestructura comercial).
- **Hardware:** El sistema debe ser capaz de operar en hardware preexistente
- **Alcance Temporal:** Ciclo de vida estricto de 16 semanas, dividido en 8 Sprints de desarrollo incremental.

4.1 Modelo de proceso

Modelo de ciclo de vida: Iterativo-incremental híbrido

El proyecto adopta un enfoque que combina la planificación temporal de Scrum con el flujo continuo de Kanban, integrado dentro de un marco de gestión predictiva conforme a la ISO/IEC/IEEE 16326:2019.

Características:

Iteraciones. - Ciclos de desarrollo de aproximadamente 2 semanas para cada sprint, con entregables incrementales al final de cada iteración.

Flujo de trabajo. - Tablero de Kanban para gestionar el estado de las actividades a realizar, con límites de trabajo en progreso para optimizar el flujo de trabajo. Sirve como mecanismo principal de transferencia de información entre los procesos técnicos y los procesos de gestión.

Reunión quincenal. - Revisión formal de avance del sprint, evaluación de cambios, ajuste de la planificación del PMP y priorización del backlog. Actúa como Sprint review y punto de control de gestión. Así mismo, al terminar el Sprint review se realiza el Sprint Planning.

Reuniones operativas. - Entre reuniones quincenales, el equipo se coordina según las necesidades para resolver problemas que van surgiendo, sin imponer reuniones innecesarias.

Gestión de cambios. - Integrada en la reunión quincenal, con análisis de impacto en alcance, cronograma y presupuesto, siguiendo el proceso de control de cambios del PMP.

Daily Scrum. - Cada día, los desarrolladores rellenan en el Daily Scrum, lo que hicieron ese día e indican si tuvieron o tienen algún problema, el problema se tratará con el Scrum Master y este deberá indicar si se ha resuelto el problema.

Sprint Planning. - Se indica cuáles serán las tareas a realizar en el siguiente sprint.

Hitos y línea base. - Al final de cada sprint, se realiza una retrospectiva donde el equipo analiza el desempeño del sprint, identifica mejoras y ajusta el proceso para el siguiente ciclo. El entregable incremental (PMP, Reportes de Sprint, backlog, etc.) se somete a una revisión que permite establecer una línea base técnica antes de pasar a la siguiente fase de construcción.

Inicio y cierre. - El modelo inicia con una fase de “inspección” para definir las historias de usuario con base en los requisitos iniciales y para definir la organización del equipo de trabajo y termina con una fase de “transición de cierre” para el despliegue final en el restaurante.

4.1.2 Justificación del modelo

Este modelo nos permite mantener la predictibilidad necesaria para la gestión formal del proyecto mientras se beneficia de la flexibilidad del flujo de Kanban para adaptarse a imprevistos sin la rigidez de las reuniones Scrum puras. El tablero Kanban proporciona visibilidad en tiempo real del estado del proyecto, facilitando la toma de decisiones en las reuniones quincenales.

“El desarrollo seguirá los estándares de codificación definidos en la sección 5.4.4 de este plan, y la calidad de los productos de trabajo se asegurará mediante la técnica de definition of Done (DoD), actuando como criterio de aceptación objetivo para cada tarea del tablero Kanban.

4.2 Mejora del proceso

Mejora del Proceso

Objetivo de la Mejora Continua

El proceso de desarrollo del proyecto incorporará actividades de mejora continua con el propósito de optimizar la calidad del software, reducir incidencias y aumentar la eficiencia operativa del equipo durante cada Sprint.

La mejora del proceso permitirá identificar problemas técnicos, operativos y organizacionales que puedan afectar el cumplimiento de objetivos, aplicando acciones correctivas y preventivas de manera iterativa.

Este enfoque se encuentra alineado con las prácticas de seguimiento, evaluación y retroalimentación propuestas en IEEE 16326 para la gestión y mejora continua de proyectos de software.

Estrategia de Mejora Continua

La mejora del proceso se realizará principalmente mediante:

- Sprint Retrospectives.
- Revisión de métricas de desempeño.
- Análisis de incidencias.
- Retroalimentación del equipo.
- Evaluación de resultados de pruebas.
- Seguimiento de errores y tiempos de respuesta.
- Validación de cumplimiento de objetivos del Sprint.

Al finalizar cada Sprint, el equipo analizará los resultados obtenidos para identificar:

- Actividades que generaron retrasos.
- Problemas de comunicación.
- Errores recurrentes.
- Cuellos de botella técnicos.
- Fallas de integración.
- Deficiencias en pruebas o despliegue.
- Desviaciones respecto a las estimaciones iniciales.

Con base en esta información se propondrán acciones de mejora para los Sprints posteriores.

Distinción de Niveles de Mejora

Las mejoras identificadas durante el desarrollo serán clasificadas según su impacto y alcance organizacional.

Mejoras Operativas del Proyecto

Corresponden a mejoras que pueden implementarse directamente dentro del proyecto sin afectar significativamente la continuidad del desarrollo.

Entre estas mejoras se incluyen:

- Ajustes de estimaciones.
- Redistribución de tareas.
- Optimización del flujo Kanban.
- Incremento de pruebas automatizadas.

- Mejoras de documentación.
- Refactorización de módulos.
- Optimización de tiempos de integración.

Estas mejoras podrán ser aplicadas inmediatamente durante el Sprint o en la siguiente iteración.

Mejoras Organizacionales

Corresponden a mejoras que impactan procesos generales, herramientas institucionales o metodologías utilizadas por la organización.

Estas mejoras podrán incluir:

- Cambios metodológicos.
- Adopción de nuevas herramientas.
- Modificación de estándares internos.
- Nuevas políticas de control de calidad.
- Estrategias institucionales de automatización.

Las mejoras organizacionales deberán ser evaluadas y aprobadas por los responsables del proyecto antes de su implementación formal.

Métricas Utilizadas para la Mejora

La mejora continua del proceso será apoyada mediante métricas ágiles y de calidad que permitan evaluar el desempeño real del proyecto.

Sprint Velocity

Permitirá medir la cantidad promedio de Story Points completados por Sprint, ayudando a estimar la capacidad real del equipo y ajustar futuras planeaciones.

Burn-down Chart

Permitirá monitorear visualmente el avance del Sprint respecto al trabajo restante y detectar desviaciones del cronograma.

Bug Rate

Ayudará a medir la cantidad de errores detectados durante las pruebas funcionales y de integración, permitiendo identificar módulos con mayor incidencia de fallos.

Cycle Time

Permitirá medir el tiempo requerido para completar tareas individuales desde el inicio de desarrollo hasta su finalización, ayudando a detectar cuellos de botella y optimizar el flujo de trabajo.

Acciones de Mejora

Cuando se detecten problemas durante el desarrollo del proyecto podrán implementarse acciones como:

- Redistribución de tareas entre integrantes.
- Ajuste de estimaciones del Sprint.
- Repriorización del Product Backlog.
- Refactorización de componentes.
- Incremento de cobertura de pruebas automatizadas.
- Optimización de integración continua.
- Mejora de documentación técnica.
- Ajustes de comunicación interna.
- Optimización del flujo de despliegue.

Cada acción de mejora deberá definir:

- Problema identificado.
- Objetivo esperado.
- Responsable de implementación.
- Métrica asociada.
- Resultado esperado.
- Criterio de éxito medible.

Criterios de Éxito Medibles

Las acciones de mejora serán evaluadas utilizando criterios cuantificables que permitan verificar su efectividad.

Algunos criterios de éxito considerados incluyen:

Acción de Mejora	Criterio de Éxito
Reducción de errores	Disminución del Bug Rate entre Sprints
Optimización de tareas	Reducción del Cycle Time promedio
Mejora de productividad	Incremento del Sprint Velocity
Mejor integración	Disminución de incidencias de despliegue
Mejora de calidad	Mayor porcentaje de pruebas aprobadas

La efectividad de cada mejora será revisada durante las Sprint Retrospectives posteriores.

Roles y Responsabilidades para la Mejora

La autoridad para aprobar, coordinar e implementar mejoras del proceso será distribuida de la siguiente manera:

Scrum Master

Responsable de:

- Identificar bloqueos.
- Coordinar retrospectivas.
- Proponer mejoras operativas.
- Dar seguimiento a métricas.
- Supervisar implementación de acciones correctivas.

Product Owner

Responsable de:

- Validar impacto funcional de las mejoras.
- Priorizar cambios relacionados con procesos del negocio.
- Aprobar modificaciones que afecten requerimientos funcionales.

Equipo de Desarrollo

Responsable de:

- Proponer mejoras técnicas.
- Implementar acciones correctivas.
- Documentar incidencias.
- Participar en revisiones retrospectivas.
- Actualizar métricas y evidencias.

Responsable del Proyecto o Cliente

Responsable de:

- Aprobar mejoras organizacionales relevantes.
- Validar cambios que afecten la operación real del restaurante.
- Revisar resultados generales del proceso de mejora.

Gestión de Datos Históricos

Con el propósito de mejorar futuras estimaciones y fortalecer la planeación organizacional, el proyecto mantendrá un registro histórico de información relevante obtenida durante cada Sprint.

La información almacenada incluirá:

- Velocity histórico.
- Burndown Charts.
- Bug Rate por Sprint.
- Cycle Time promedio.
- Incidencias registradas.
- Resultados de pruebas.
- Cambios de alcance.
- Retrasos identificados.
- Ajustes de estimación.
- Evidencias de retrospectivas.

Estos datos permitirán:

- Mejorar futuras estimaciones.
- Identificar patrones de riesgo.
- Optimizar procesos de desarrollo.
- Generar métricas comparativas.
- Incrementar la precisión de planeación en proyectos posteriores.

Toda la información histórica será almacenada dentro de los repositorios documentales y herramientas de seguimiento del proyecto para mantener trazabilidad y respaldo institucional.

Seguimiento de la Mejora

El seguimiento de las actividades de mejora será realizado mediante:

- Sprint Retrospectives.
- Reuniones de seguimiento.
- Actualización del tablero Kanban.
- Revisión de métricas.
- Reportes de pruebas automatizadas.
- Validaciones funcionales.
- Registro de incidencias y acciones correctivas.

La aplicación continua de estas actividades permitirá incrementar progresivamente la estabilidad, calidad, mantenibilidad y eficiencia del sistema durante todo el ciclo de vida del proyecto.

4.3 Plan de Infraestructura

Infraestructura de Trabajo

El desarrollo del proyecto será realizado principalmente utilizando los dispositivos personales de cada integrante del equipo, previamente configurados con las herramientas, dependencias y entornos necesarios para garantizar compatibilidad durante el ciclo de desarrollo.

Cada estación de trabajo contará con:

- Entorno Docker configurado.
- Herramientas de desarrollo instaladas.
- Acceso al repositorio GitHub.
- Dependencias del proyecto actualizadas.
- Acceso a herramientas de pruebas y documentación.

En caso de requerirse trabajo colaborativo presencial, integración conjunta o validaciones funcionales, el equipo utilizará cubículos y salones disponibles dentro de la institución educativa para realizar reuniones técnicas, pruebas integradas y actividades de coordinación.

La utilización de infraestructura distribuida permite mantener continuidad de desarrollo y flexibilidad operativa durante los Sprints del proyecto.

Infraestructura Tecnológica

Trello

Se utilizará **Trello** como herramienta de administración visual de tareas mediante metodología Kanban.

La plataforma permitirá:

- Gestión de actividades del Sprint.
- Seguimiento de tareas.
- Priorización de actividades.
- Control del flujo de trabajo.
- Monitoreo del avance del proyecto.

Trello fue seleccionado debido a su facilidad de uso, colaboración en tiempo real y compatibilidad multiplataforma.

GitHub

Se utilizará **GitHub** como repositorio centralizado para el almacenamiento, control de versiones y respaldo del código fuente y documentación del proyecto.

GitHub permitirá:

- Gestión de ramas y versiones.
- Integración continua.
- Respaldo histórico del código.
- Colaboración simultánea entre integrantes.
- Control de cambios y trazabilidad.

La plataforma fue seleccionada debido a su amplia adopción en entornos colaborativos y compatibilidad con herramientas DevOps y automatización.

Docker

Se implementará **Docker** para estandarizar el entorno de desarrollo y ejecución entre todos los integrantes del equipo.

Docker permitirá:

- Aislamiento de dependencias.
- Uniformidad del entorno.
- Reducción de problemas de compatibilidad.
- Portabilidad del sistema.
- Facilidad de despliegue y pruebas.

La utilización de contenedores permitirá asegurar que todos los desarrolladores trabajen sobre configuraciones equivalentes independientemente del sistema operativo utilizado.

OneDrive

Se utilizará **OneDrive** como sistema complementario de almacenamiento y respaldo de documentación técnica, evidencias y archivos administrativos del proyecto.

La plataforma permitirá mantener disponibilidad y respaldo de información relevante durante todo el ciclo de desarrollo.

Red Local Wi-Fi del Restaurante

La red local del restaurante será utilizada para permitir la comunicación entre:

- Tablets de pedidos.
- Módulo de caja.

- Sistema de cocina.
- Dispositivos administrativos.
- Servidor local del sistema.

Esta infraestructura permitirá validar sincronización de pedidos, actualización de estados y comunicación en tiempo real entre los distintos módulos operativos del sistema.

Herramientas de Ingeniería

Visual Studio Code

Se utilizará **Visual Studio Code** como entorno principal de desarrollo debido a su compatibilidad con múltiples lenguajes, integración con Git y soporte para extensiones de desarrollo y depuración.

OpenCode

Se empleará **OpenCode** para la ejecución de pruebas y validaciones funcionales del sistema durante el proceso de desarrollo.

La herramienta permitirá verificar comportamiento funcional y estabilidad operativa del sistema antes de las validaciones finales.

Composer y npm

Se utilizarán **Composer** y **npm** para la administración de dependencias backend y frontend respectivamente.

Estas herramientas permitirán:

- Instalación automatizada de bibliotecas.
- Gestión de versiones de dependencias.
- Actualización controlada de paquetes.
- Compatibilidad entre entornos.

MySQL

Se implementará **MySQL** como sistema gestor de base de datos principal del proyecto.

MySQL fue seleccionado debido a:

- Estabilidad operativa.
- Compatibilidad con PHP.
- Buen rendimiento en entornos locales.
- Facilidad de integración con Docker.

Sistemas Habilitadores

Control de Versiones

GitHub permitirá mantener:

- Control de versiones.
- Integración continua.
- Respaldo histórico del proyecto.
- Gestión de ramas y cambios.

Contenerización

Docker permitirá que todos los integrantes trabajen sobre una misma configuración de entorno, reduciendo problemas relacionados con dependencias, versiones y configuración local.

Comunicación en Red

La red local del restaurante permitirá sincronizar en tiempo real:

- Pedidos.
- Estados de órdenes.
- Cobros.
- Actualizaciones entre módulos.
- Comunicación entre cocina, caja y tablets.

Responsables de Infraestructura

Docker — Jesús Vicente Ríos Carrera

Responsable de:

- Mantener actualizadas las imágenes Docker.
- Verificar el funcionamiento de contenedores.
- Supervisar la configuración del entorno.
- Garantizar acceso a la versión más reciente del sistema.

Trello — George Edmundo Osornio Tapia y Guadalupe Yosabeth Pérez Estrada

Responsables de:

- Actualizar tareas del Sprint.
- Administrar prioridades.
- Supervisar el flujo de trabajo.
- Mantener seguimiento del tablero Kanban.

OpenCode — Oswaldo García Ramírez

Responsable de:

- Coordinar pruebas funcionales.
- Validar integraciones del sistema.
- Supervisar el entorno de pruebas.
- Verificar estabilidad operativa.

GitHub — Todo el Equipo

Responsables de:

- Mantener actualizado el repositorio.
- Gestionar ramas y merges.
- Documentar cambios relevantes.
- Mantener respaldo y trazabilidad del código fuente.

Gestión y Mantenimiento de Infraestructura

La infraestructura tecnológica será revisada periódicamente durante los Sprints para garantizar:

- Disponibilidad de herramientas.
- Compatibilidad entre entornos.
- Actualización de dependencias.
- Estabilidad de servicios.
- Correcta sincronización de repositorios.
- Funcionamiento adecuado de contenedores y red local.

Cualquier incidencia relacionada con infraestructura deberá registrarse y resolverse mediante los mecanismos definidos dentro del proceso de mejora continua del proyecto.

4.4 Métodos, herramientas y técnicas

Metodologías de Desarrollo

Para el desarrollo del proyecto se adoptó una combinación de las metodologías **Scrum** y **DevOps**, debido a que permiten una gestión iterativa del trabajo y una integración continua entre desarrollo, pruebas y despliegue.

La metodología Scrum fue seleccionada porque facilita la organización del proyecto mediante sprints, backlog, reuniones de seguimiento y retrospectivas, permitiendo una administración flexible de cambios y prioridades. Además, favorece la trazabilidad de actividades y la colaboración entre los integrantes del equipo.

Por otro lado, DevOps fue implementado para automatizar procesos de integración, pruebas y despliegue utilizando contenedores Docker y herramientas de automatización. Esta metodología permitió reducir errores de configuración, mejorar la estabilidad del entorno y acelerar el ciclo de retroalimentación.

La combinación de Scrum y DevOps fue elegida debido a que proporciona mayor adaptabilidad y control del proyecto respecto a metodologías tradicionales secuenciales, las cuales presentan menor flexibilidad ante cambios frecuentes en requisitos o funcionalidades.

Herramientas Utilizadas

Sistema Operativo — Fedora 44

Se utilizó **Fedora 44 (Forty Four)** como sistema operativo principal debido a su estabilidad, compatibilidad con herramientas modernas de desarrollo y soporte actualizado para tecnologías de contenedores y automatización.

La versión Fedora 44 fue seleccionada porque incorpora versiones recientes del kernel Linux, compatibilidad estable con Docker Engine 29 y soporte optimizado para Node.js 22 y herramientas DevOps modernas. No se eligieron versiones anteriores debido a limitaciones de compatibilidad y soporte menos actualizado, mientras que versiones posteriores podrían presentar características experimentales o inestabilidad en determinadas herramientas.

XAMPP 8.0.30-0

Se empleó **XAMPP 8.0.30-0** como entorno legacy de desarrollo local para mantener compatibilidad con módulos previamente desarrollados en PHP y MariaDB.

La versión utilizada fue seleccionada por ofrecer estabilidad y compatibilidad con el código heredado del sistema. Versiones anteriores presentaban menor soporte para bibliotecas modernas, mientras que versiones posteriores podían generar incompatibilidades con componentes legacy existentes.

Docker Engine 29.4.2

Se utilizó **Docker Engine 29.4.2** para la contenerización del sistema y la estandarización del entorno de ejecución.

Esta versión fue seleccionada debido a su estabilidad, mejoras de rendimiento y compatibilidad con Docker Compose. Su implementación permitió aislar dependencias, garantizar portabilidad y reducir conflictos entre entornos de desarrollo y producción.

No se utilizaron versiones anteriores debido a limitaciones en soporte de redes y volúmenes, mientras que versiones posteriores aún no contaban con suficiente validación en el entorno del proyecto.

Docker Compose 5.1.2

Se implementó **Docker Compose 5.1.2** para la orquestación de servicios y contenedores del sistema.

La herramienta fue seleccionada debido a que permite administrar múltiples servicios mediante archivos de configuración centralizados, simplificando el despliegue y mantenimiento del sistema.

La versión 5.1.2 fue elegida por su compatibilidad directa con Docker Engine 29.4.2 y por ofrecer una configuración más estable respecto a versiones anteriores.

PHP 8.2

Se utilizó **PHP 8.2** como lenguaje backend principal dentro del entorno Docker. La versión 8.2 fue seleccionada debido a las mejoras de rendimiento, seguridad y tipado respecto a PHP 8.0 y versiones anteriores. Asimismo, proporciona mayor compatibilidad con bibliotecas modernas y herramientas actuales de pruebas automatizadas.

No se utilizaron versiones más antiguas debido a limitaciones de rendimiento y soporte, mientras que versiones posteriores no habían sido completamente estabilizadas al momento del desarrollo.

MySQL 8.0

Se implementó **MySQL 8.0** como sistema gestor de base de datos dentro del entorno Docker.

Esta versión fue seleccionada por ofrecer mejoras en rendimiento, optimización de consultas y seguridad respecto a versiones anteriores. Además, mantiene compatibilidad con herramientas de contenerización y administración modernas.

Node.js 22.22.2

Se utilizó **Node.js 22.22.2** como entorno de ejecución JavaScript para herramientas auxiliares y automatización.

La versión 22 fue seleccionada debido a su soporte LTS, mejoras de rendimiento y compatibilidad con herramientas modernas como Playwright y servidores MCP.

No se eligieron versiones anteriores debido a limitaciones en dependencias y compatibilidad con herramientas recientes de automatización.

npm 10.9.7

Se empleó **npm 10.9.7** como gestor de paquetes para la instalación y administración de dependencias JavaScript del proyecto.

Esta versión fue seleccionada por mantener compatibilidad directa con Node.js 22 y proporcionar mejoras en resolución de dependencias y rendimiento.

Playwright 1.59.1

Se implementó **Playwright 1.59.1** para automatizar pruebas End-to-End del sistema.

La herramienta fue seleccionada debido a su capacidad para automatizar navegadores modernos, capturar evidencia visual y validar flujos completos de usuario en diferentes módulos del sistema.

La versión utilizada ofrece mejor compatibilidad con Chromium y mayor estabilidad respecto a versiones anteriores.

PHPUnit 10.5

Se utilizó **PHPUnit 10.5** para la ejecución de pruebas unitarias sobre componentes backend desarrollados en PHP.

La versión 10.5 fue seleccionada por ser compatible con PHP 8.2 y proporcionar mejoras en cobertura de pruebas, validación de errores y rendimiento respecto a versiones previas.

TestSprite MCP 0.0.37

Se implementó **TestSprite MCP 0.0.37** como motor de pruebas automatizadas asistidas por inteligencia artificial.

Esta herramienta fue seleccionada debido a su capacidad para generar planes de prueba automáticos, ejecutar validaciones en la nube y generar reportes interactivos con métricas, capturas y videos.

La versión utilizada fue elegida por mantener compatibilidad con cline y Playwright MCP dentro del entorno configurado del proyecto.

Markdownify MCP 1.0.4

Se utilizó **Markdownify MCP 1.0.4** para convertir reportes HTML y PDF generados durante las pruebas automatizadas a formato Markdown.

Esta herramienta permitió mantener documentación homogénea y facilitar la integración de reportes dentro de los artefactos del proyecto.

Git

Se implementó **Git** como sistema de control de versiones para gestionar cambios, ramas y respaldos del código fuente.

Git fue seleccionado debido a su amplia adopción en entornos de desarrollo colaborativo y su capacidad para mantener trazabilidad de cambios, historial de versiones y recuperación de código.

Técnicas Utilizadas

Contenerización

Se aplicó la técnica de contenerización mediante Docker para garantizar portabilidad, aislamiento de dependencias y uniformidad entre entornos de desarrollo y producción.

Automatización de pruebas

Se utilizaron técnicas de automatización de pruebas mediante Playwright, PHPUnit y TestSprite MCP para validar funcionalidades críticas del sistema y reducir errores durante el proceso de desarrollo.

Integración continua

Se implementaron prácticas de integración continua mediante Git, Docker y pruebas automatizadas, permitiendo validar cambios frecuentes y mantener estabilidad en el proyecto.

Control de versiones

Se aplicó control de versiones utilizando Git para registrar cambios del sistema, facilitar la colaboración del equipo y mantener respaldo histórico de modificaciones.

Pruebas End-to-End (E2E)

Se utilizaron pruebas End-to-End mediante Playwright para simular escenarios reales de interacción de usuarios con el sistema, incluyendo autenticación, pedidos, cocina y caja.

Pruebas unitarias

Se aplicaron pruebas unitarias con PHPUnit para validar el funcionamiento individual de módulos backend y detectar errores en etapas tempranas del desarrollo.

4.4.1 Herramientas del Gestion y seguimiento - Trello

Uso del Tablero Kanban

Se utilizará la herramienta **Trello** como plataforma de gestión visual de tareas mediante la metodología Kanban, debido a su facilidad para organizar actividades, monitorear avances y mantener trazabilidad durante el desarrollo del proyecto.

El tablero Kanban permitirá:

- Visualizar el estado actual de las actividades del Sprint.
- Asignar responsables a cada tarea.
- Establecer prioridades de trabajo.
- Controlar el flujo de actividades durante el ciclo de desarrollo.
- Registrar comentarios y seguimiento de incidencias.
- Mantener evidencia del progreso del proyecto.

La selección de Trello se realizó debido a su facilidad de uso, accesibilidad multiplataforma y capacidad de colaboración en tiempo real. Asimismo, permite mantener una administración organizada de tareas sin requerir infraestructura adicional compleja.

Durante cada Sprint se manejará un promedio aproximado de 10 tareas activas, distribuidas según la complejidad funcional y la carga de trabajo del equipo de desarrollo.

Cada integrante trabajará normalmente entre 3 y 4 tareas simultáneas, dependiendo de la prioridad, criticidad y duración estimada de cada actividad.

Cada tarjeta dentro del tablero Kanban deberá contener como mínimo:

- Responsable asignado.
- Nivel de prioridad.
- Estado actual de la tarea.
- Descripción funcional.
- Comentarios de seguimiento.
- Evidencia o referencias relacionadas.
- Fecha estimada de finalización.

Definition of Done (DoD)

Para considerar una tarea como finalizada dentro del Sprint, deberá cumplir obligatoriamente con los siguientes criterios de aceptación:

- Código implementado y funcional.
- Integración correcta dentro del repositorio GitHub.
- Revisión de código entre integrantes del equipo.
- Validación funcional por parte del Scrum Master o Product Owner.

- Pruebas funcionales ejecutadas y aprobadas.
- Validación dentro del entorno Docker o herramientas de ejecución configuradas.
- Actualización del estado de la tarea en Trello.
- Registro de evidencias y observaciones relevantes.

Únicamente después de cumplir todos los criterios establecidos en la Definition of Done (DoD), la actividad podrá marcarse como completada dentro del tablero Kanban. En caso de que el desarrollador responsable no actualice la tarea, el Scrum Master o Product Owner podrá realizar la actualización correspondiente después de verificar la correcta implementación y validación de la funcionalidad.

4.5 Plan de aceptación del producto

Criterios de Aceptación

El sistema será considerado aceptado cuando cumpla satisfactoriamente con los requisitos funcionales, operativos y de rendimiento definidos para el entorno del restaurante. Como criterio principal de aceptación, el sistema deberá ser capaz de procesar un pedido desde la tablet asignada a la mesa, registrar la información, enviarla automáticamente a cocina y caja, procesar el cobro correspondiente y actualizar el estado de la orden en un tiempo menor a 2 segundos bajo condiciones normales de operación dentro de la red local del establecimiento.

$T < 2$ segundos

El tiempo de respuesta fue definido considerando la necesidad operativa de mantener un flujo continuo de atención al cliente y evitar retrasos durante la toma y procesamiento de pedidos.

Asimismo, el sistema deberá garantizar:

- Correcta sincronización entre módulos.
- Disponibilidad de información en tiempo real.
- Persistencia adecuada de datos.
- Integridad de pedidos y transacciones.
- Estabilidad operativa durante pruebas continuas.

Métodos de Aceptación

La aceptación del sistema se realizará mediante la aplicación de diferentes métodos de validación funcional y operativa, incluyendo:

- Pruebas funcionales de módulos.
- Demostración operativa del sistema.
- Inspección de funcionalidades administrativas.
- Validación de procesos de caja y cocina.
- Revisión de manuales técnicos y de usuario.
- Simulación de escenarios reales de operación.
- Validación de rendimiento dentro de la red local.

Estos métodos permitirán verificar que el sistema cumple con los requisitos establecidos y que su comportamiento es consistente dentro del entorno de trabajo esperado.

Plan de Pruebas de Aceptación

Se realizará una jornada de validación denominada “Prueba en Vivo”, en la cual se simulará el flujo operativo completo del restaurante para comprobar el funcionamiento integral del sistema antes de emitir el acta final de aceptación.

La validación se ejecutará utilizando los siguientes recursos:

- Tablets Android.
- Laptops con Windows.
- Dispositivos móviles de prueba.
- Red local Wi-Fi del entorno operativo.
- Entorno Docker configurado para ejecución del sistema.

Durante esta fase se evaluará:

- Generación de pedidos.
- Modificación de órdenes activas.
- Envío automático de pedidos a cocina.
- Sincronización entre módulos.
- Cobro y registro de pagos en caja.
- Actualización automática de estados.
- Impresión de tickets.
- Liberación y cierre de pedidos.
- Integridad de información almacenada.
- Tiempo de respuesta del sistema.

Acuerdo Formal de Aceptación

Los criterios y métodos de aceptación serán revisados y aprobados conjuntamente entre el equipo de desarrollo y el responsable operativo del restaurante antes de iniciar las pruebas finales del sistema.

Cualquier observación, incidencia o solicitud de ajuste identificada durante la validación deberá registrarse formalmente y resolverse antes de la aprobación definitiva del producto.

Evidencia de Aceptación

La aceptación final del sistema quedará respaldada mediante la siguiente documentación y evidencia técnica:

- Reportes de pruebas funcionales.
- Registro de incidencias detectadas.
- Resultados de validación operativa.
- Capturas y evidencias de funcionamiento.
- Reportes generados por herramientas de prueba.

- Validación de tiempos de respuesta.
- Acta de conformidad y visto bueno del responsable del restaurante.

Toda la evidencia recopilada será almacenada como parte de la documentación oficial del proyecto para asegurar trazabilidad y respaldo del proceso de aceptación.

4.6 Organización del proyecto

Estructura Organizacional

La organización del proyecto se definió utilizando una estructura modular basada en funcionalidades, con el propósito de facilitar el desarrollo, mantenimiento, integración y seguimiento de actividades durante el ciclo de vida del software.

El proyecto se desarrollará bajo un esquema colaborativo utilizando la metodología Scrum, permitiendo que las responsabilidades y actividades puedan ajustarse dinámicamente durante cada Sprint según las necesidades operativas, prioridades del backlog y carga de trabajo del equipo.

Aunque las responsabilidades pueden variar durante el desarrollo, cada integrante trabajará principalmente sobre un módulo funcional específico para mantener trazabilidad, especialización técnica y control de cambios.

La división modular del sistema permite:

- Reducir dependencias entre componentes.
- Facilitar pruebas e integración continua.
- Mejorar la mantenibilidad del sistema.
- Distribuir eficientemente las tareas del Sprint.
- Incrementar la trazabilidad de cambios y errores.

Organización por Módulos Funcionales

Módulo de Autenticación y Control de Acceso

Este módulo corresponde al sistema de autenticación de usuarios y control de acceso al sistema.

Su función principal es validar la identidad de cada usuario mediante credenciales de acceso asociadas a un rol específico dentro del restaurante.

Cada usuario deberá autenticarse utilizando:

- Nombre de usuario o identificador.
- Contraseña asignada.
- Rol correspondiente dentro del sistema.

El módulo controlará el acceso a funcionalidades específicas según el perfil autorizado, permitiendo restringir operaciones de acuerdo con el nivel de permisos definido para cada rol.

Los roles considerados dentro del sistema incluyen:

- Administrador.
- Mesero.
- Cocina.
- Caja.

La implementación de este módulo fue separada del resto del sistema debido a que representa un componente crítico de seguridad y control operativo, permitiendo centralizar la validación de sesiones, permisos y autenticación.

Asimismo, esta separación modular facilita:

- El mantenimiento del sistema.
- La escalabilidad futura.
- La administración de permisos.
- La integración con otros módulos funcionales.
- La validación independiente mediante pruebas unitarias y funcionales.

Asignación de Responsabilidades

Durante cada Sprint, las actividades serán distribuidas entre los integrantes del equipo considerando:

- Complejidad técnica.
- Prioridad funcional.
- Dependencias entre módulos.
- Tiempo estimado de desarrollo.
- Carga de trabajo actual.

La asignación de tareas será gestionada mediante el tablero Kanban del proyecto, permitiendo mantener seguimiento continuo del avance, estado y responsables de cada actividad.

En caso de requerirse soporte o integración entre módulos, los integrantes podrán colaborar temporalmente en componentes distintos a su módulo principal, manteniendo coordinación mediante reuniones de seguimiento y revisión del Sprint.

4.6.1 Interfaces externas

Participación de Actores Externos

El proyecto contempla la participación de actores externos con el propósito de validar requisitos, verificar el funcionamiento operativo del sistema y asegurar que las funcionalidades desarrolladas cumplan con las necesidades reales del entorno del restaurante.

La interacción con estos participantes permitirá obtener retroalimentación funcional y operativa durante las diferentes fases del proyecto, especialmente en actividades de validación y aceptación del sistema.

Dueño del Restaurante

El dueño del restaurante actuará como principal responsable de la validación de requisitos funcionales y operativos del sistema.

Sus responsabilidades dentro del proyecto incluyen:

- Validar necesidades operativas del restaurante.
- Revisar y aprobar requerimientos funcionales.
- Verificar que los módulos desarrollados cumplan con los procesos reales del negocio.
- Participar en revisiones de avance y validación de funcionalidades.
- Aprobar criterios de aceptación del producto.
- Emitir la conformidad final del sistema previo a su liberación.

La participación del responsable del restaurante es fundamental para asegurar que el sistema mantenga alineación con los procesos administrativos y operativos reales del entorno de trabajo.

Usuarios Finales

Los usuarios finales participarán como consultores funcionales durante las pruebas de usabilidad y validación operativa del sistema.

Su intervención permitirá evaluar el comportamiento del sistema en escenarios reales de uso, verificando la facilidad de operación, tiempos de respuesta y comprensión de las interfaces.

Los usuarios finales colaborarán principalmente en:

- Pruebas de usabilidad.
- Simulación de flujos operativos reales.
- Validación de procesos de pedidos.
- Verificación de funciones de cocina y caja.
- Evaluación de interacción con tablets y dispositivos móviles.
- Identificación de errores o mejoras operativas.

La retroalimentación proporcionada por los usuarios finales será utilizada para realizar ajustes funcionales y mejorar la experiencia de uso antes de la aceptación definitiva del producto.

Comunicación y Retroalimentación

La comunicación con actores externos se realizará mediante reuniones de revisión, demostraciones operativas y sesiones de validación funcional durante los Sprints definidos en el proyecto.

Toda observación, incidencia o recomendación identificada durante estas actividades será registrada y documentada para mantener trazabilidad y control de cambios dentro del proceso de desarrollo.

4.6.2 Interfaces internas

Comunicación Interna del Proyecto

Las interfaces internas del proyecto corresponden a los mecanismos de coordinación, comunicación y colaboración establecidos entre los integrantes del equipo de desarrollo para asegurar la correcta ejecución de actividades durante cada Sprint.

La comunicación interna se realizará mediante reuniones Scrum, seguimiento en tablero Kanban y coordinación técnica continua entre los responsables de cada módulo funcional.

Estas interfaces permiten:

- Coordinar actividades de desarrollo.
- Gestionar dependencias entre módulos.
- Resolver bloqueos técnicos y operativos.
- Mantener trazabilidad de tareas.
- Validar avances funcionales.
- Facilitar la integración continua del sistema.

Scrum Master

El Scrum Master será responsable de facilitar la comunicación entre los integrantes del equipo y asegurar el cumplimiento de la metodología Scrum durante el desarrollo del proyecto.

Sus principales responsabilidades incluyen:

- Coordinar reuniones Daily Scrum.
- Identificar y eliminar bloqueos técnicos u operativos.
- Supervisar el avance de actividades del Sprint.
- Mantener actualizado el flujo de trabajo en el tablero Kanban.
- Facilitar la colaboración entre integrantes.
- Verificar el cumplimiento de los procesos definidos.

El Scrum Master actuará como facilitador del equipo, promoviendo la organización y continuidad del desarrollo.

Product Owner

El Product Owner actuará como enlace principal entre el dueño del restaurante y el equipo de desarrollo.

Sus responsabilidades incluyen:

- Gestionar y priorizar el Product Backlog.
- Validar requerimientos funcionales.
- Comunicar necesidades operativas al equipo técnico.
- Revisar funcionalidades desarrolladas.
- Participar en validaciones y revisiones de Sprint.
- Aprobar entregables funcionales.

El Product Owner garantizará que las funcionalidades implementadas mantengan alineación con los objetivos operativos y administrativos del restaurante.

Desarrolladores Frontend y Backend

Los desarrolladores frontend y backend mantendrán coordinación técnica continua para asegurar la correcta integración entre interfaces visuales, lógica de negocio y servicios del sistema.

La comunicación diaria entre desarrolladores permitirá:

- Integrar funcionalidades entre módulos.
- Validar flujos operativos del sistema.
- Sincronizar cambios de interfaz y backend.
- Resolver errores de integración.
- Garantizar compatibilidad entre componentes.
- Mantener consistencia en la experiencia de usuario.

La coordinación técnica será especialmente importante en los módulos relacionados con:

- Tablets de pedidos.
- Sistema de autenticación.
- Cocina.
- Caja y pagos.
- Gestión de órdenes.
- Sincronización en tiempo real.

Mecanismos de Coordinación

La coordinación interna del proyecto se realizará mediante:

- Reuniones Daily Scrum.
- Sprint Planning.
- Sprint Review.
- Sprint Retrospective.
- Tablero Kanban en Trello.
- Control de versiones con Git.
- Comunicación técnica entre responsables de módulos.

Estos mecanismos permitirán mantener seguimiento continuo del estado del proyecto y asegurar la correcta integración de los componentes desarrollados.

4.6.3 Autoridades y Responsabilidades

Para asegurar que el trabajo se realice según lo planeado, se asignan los módulos mencionados con anterioridad a roles específicos mediante una Matriz de Responsabilidades lo que nos permite identificar quien ejecuta la actividad, quien aprueba el trabajo realizado, quién debe ser consultado y quién debe mantenerse informado sobre el avance del módulo.

Actividad / Módulo	Responsable	Rinde cuentas / Aprueba	Consultado	Informado
Autenticación (Login)	Jesús Vicente Rios Carrera	Scrum Master	QA	Dueño del restaurante
Gestión de Mesas	Uriel Vargas Angeles	Scrum Master	Meseros	Dueño del restaurante
Menú Digital	Oswaldo García Ramirez	Product Owner	Dueño del restaurante	QA
Módulo de Meseros	Uriel Vargas Angeles	Scrum Master	Meseros	QA
Módulo de Cocina	Jesús Vicente Rios Carrera	Scrum Master	Cocina / Chef	QA
Módulo de Caja	Oswaldo García Ramirez	Product Owner	Dueño del restaurante	Caja

Panel de Administración	Uriel Vargas Angeles	Product Owner	Dueño del restaurante	QA
Inventario y Stock	Jesús Vicente Rios Carrera	Product Owner	Administración	QA
Integración de módulos	Equipo de desarrolladores	Scrum Master	Product Owner	Dueño del restaurante
Pruebas funcionales	Oswaldo García Ramirez	Scrum Master	QA	Equipo de desarrollo
Validación del flujo completo	Scrum Master y Product Owner	Product Owner	Dueño del restaurante	Equipo de desarrollo
Gestión de Docker y entorno	Jesús Vicente Rios Carrera	Scrum Master	Equipo de desarrollo	Product Owner
Gestión de Trello y seguimiento	Scrum Master y Product Owner	Scrum Master y Product Owner	Equipo de desarrollo	Dueño del restaurante
Gestión de documentación	Scrum Master y Product Owner	Scrum Master y Product Owner	Equipo de desarrollo	Product Owner

Cada responsable deberá garantizar que las funcionalidades asignadas cumplan con los criterios de aceptación, se integren correctamente al sistema y pasen por revisión de código, pruebas funcionales y validaciones antes de considerar finalizadas.

La asignación de responsabilidades podrá ajustarse entre sprints dependiendo de la carga de trabajo y necesidades del proyecto.

5. Planificación del proyecto

5.1 Inicialización del proyecto

Inicio del Proyecto

La inicialización del proyecto corresponde a la fase en la cual se definieron los objetivos generales, alcance funcional, recursos técnicos y organización inicial del sistema de comanda digital.

Durante esta etapa se establecieron las bases operativas y metodológicas necesarias para garantizar un desarrollo estructurado y alineado con las necesidades del restaurante.

La fase de inicialización permitió:

- Definir el alcance preliminar del sistema.
- Identificar requerimientos funcionales y operativos.
- Establecer la metodología de trabajo.
- Seleccionar herramientas y tecnologías.
- Asignar roles y responsabilidades.
- Estimar tiempos y esfuerzo del proyecto.
- Definir el entorno de desarrollo y pruebas.

Definición del Alcance

Durante la inicialización se definió que el sistema permitiría gestionar digitalmente el flujo operativo del restaurante mediante módulos interconectados para:

- Gestión de pedidos.
- Autenticación de usuarios.
- Administración de mesas.
- Cocina.
- Caja y pagos.
- Control de estados de órdenes.
- Sincronización en tiempo real.

El alcance fue delimitado considerando las necesidades operativas del restaurante y los recursos disponibles para el desarrollo.

Identificación de Requerimientos

En esta fase se realizaron reuniones iniciales con el responsable del restaurante para identificar procesos operativos, necesidades funcionales y problemas existentes dentro del flujo tradicional de atención.

Como resultado, se definieron requerimientos relacionados con:

- Rapidez en toma de pedidos.

- Reducción de errores manuales.
- Sincronización entre cocina y caja.
- Control de usuarios mediante roles.
- Visualización de estados de pedidos.
- Automatización de procesos operativos.

Los requerimientos obtenidos fueron registrados y organizados dentro del Product Backlog inicial.

Selección de Metodología

Se seleccionó la metodología Scrum complementada con prácticas DevOps debido a la necesidad de mantener un desarrollo iterativo, flexible y con integración continua.

La metodología Scrum fue elegida para organizar el trabajo mediante Sprints y facilitar el seguimiento de actividades, mientras que DevOps permitió automatizar pruebas, despliegues y validaciones mediante herramientas de contenerización y automatización.

Configuración del Entorno de Desarrollo

Durante la inicialización se preparó el entorno técnico necesario para el desarrollo del sistema, incluyendo:

- Instalación y configuración de Fedora 44.
- Configuración de Docker y Docker Compose.
- Preparación del entorno PHP y Node.js.
- Configuración de base de datos MySQL.
- Inicialización del repositorio Git.
- Configuración de herramientas de pruebas automatizadas.
- Preparación de entorno Docker para validaciones funcionales.

Estas configuraciones permitieron garantizar uniformidad y estabilidad en el proceso de desarrollo.

Organización del Equipo

El equipo de trabajo fue organizado bajo una estructura colaborativa, asignando responsabilidades iniciales para:

- Scrum Master.
- Product Owner.
- Desarrollo Frontend.
- Desarrollo Backend.

- Validación y pruebas.

La asignación inicial de responsabilidades permitió distribuir módulos funcionales y mantener control del avance del proyecto desde las primeras etapas.

Estimación Inicial

Durante la inicialización también se realizó una estimación preliminar del esfuerzo y tiempo requerido para el proyecto.

La estimación fue calculada considerando:

- Número de integrantes.
- Horas efectivas de desarrollo.
- Duración de Sprints.
- Complejidad funcional.
- Actividades de pruebas y documentación.

El esfuerzo total estimado del proyecto fue definido en 840 horas-hombre distribuidas entre las diferentes fases del ciclo de vida del software.

$$E = 3 \times 4 \times 70 = 840$$

Validación Inicial

Antes de iniciar el desarrollo formal, los objetivos, requerimientos y alcance del sistema fueron revisados junto con el responsable del restaurante para asegurar viabilidad técnica y alineación con las necesidades operativas identificadas.

La aprobación inicial permitió dar inicio formal al proyecto y comenzar la planificación de los primeros Sprints.

5.1.1 Plan de estimación

Objetivo del Plan de Estimación

El presente plan de estimación tiene como objetivo definir los mecanismos, técnicas y métricas utilizadas para calcular el esfuerzo, tiempo y recursos necesarios para el desarrollo del sistema de comanda digital.

La estimación permitirá establecer una planeación inicial del proyecto, así como realizar ajustes progresivos conforme avance el desarrollo y se obtenga información histórica del desempeño del equipo.

El proceso de estimación se encuentra alineado con las prácticas de planificación y seguimiento propuestas por el estándar IEEE 16326 para proyectos de software.

Técnica de Estimación

Planning Poker

La técnica principal de estimación utilizada será **Planning Poker**, debido a que permite realizar estimaciones colaborativas mediante consenso entre los integrantes del equipo de desarrollo.

Esta técnica fue seleccionada porque facilita la evaluación conjunta de:

- Complejidad técnica.
- Tiempo aproximado de implementación.
- Riesgos asociados.
- Dependencias funcionales.
- Nivel de incertidumbre.
- Esfuerzo requerido para pruebas e integración.

Durante las sesiones de estimación, cada integrante asignará valores relativos de esfuerzo a las historias de usuario utilizando Story Points.

La utilización de Planning Poker fue preferida sobre métodos tradicionales de estimación fija debido a que permite mejorar progresivamente la precisión conforme avanza el proyecto y se recopilan datos reales del desempeño del equipo.

Unidad de Medición del Esfuerzo

La unidad principal utilizada para medir el esfuerzo técnico serán los **Story Points**, los cuales representan una medida relativa de complejidad y trabajo requerido para completar una historia de usuario.

Los Story Points consideran simultáneamente:

- Tiempo estimado.
- Complejidad funcional.
- Riesgo técnico.
- Dependencias.
- Dificultad de integración.
- Incertidumbre operativa.

El uso de Story Points permite mantener flexibilidad en la planeación y evitar estimaciones rígidas basadas únicamente en horas de trabajo.

Estimación de Tiempo del Proyecto

Las estimaciones generales del proyecto fueron definidas considerando distintos escenarios de incertidumbre y disponibilidad operativa del equipo.

Escenario	Tiempo Estimado
Optimista	4 meses
Más probable	5 meses
Pesimista	6 meses

Estas estimaciones consideran:

- Disponibilidad parcial de los integrantes.
- Carga académica.
- Riesgos técnicos.
- Actividades de integración.
- Tiempo de pruebas y validaciones.
- Configuración de despliegue.
- Posibles incidencias de compatibilidad.

La definición de múltiples escenarios permite mantener márgenes de contingencia y mejorar la gestión del cronograma.

Estimación del Esfuerzo Total

El esfuerzo total del proyecto fue calculado considerando el número de integrantes, horas efectivas de desarrollo y duración estimada del proyecto.

La ecuación utilizada fue:

$$E = P \times H \times D$$

Donde:

- E = Esfuerzo total estimado en horas-hombre.
- P = Número de integrantes del equipo.
- H = Horas efectivas de trabajo por día.
- D = Días hábiles de desarrollo.

Sustituyendo los valores definidos para el proyecto:

$$E = 3 \times 4 \times 70$$

Resultado:

$$E = 840 \text{ horas-hombre}$$

La estimación considera únicamente horas efectivas de desarrollo y no contempla tiempos muertos, interrupciones externas o actividades ajenas al proyecto.

Distribución del Esfuerzo

El esfuerzo estimado fue distribuido entre las diferentes fases del ciclo de vida del software considerando la complejidad y carga de trabajo asociada a cada actividad.

Categoría	Porcentaje	Horas Estimadas
Gestión y documentación	25%	210 horas
Arquitectura y diseño	15%	126 horas
Construcción	40%	336 horas
Pruebas y QA	15%	126 horas
Despliegue y cierre	5%	42 horas

La distribución fue definida considerando que la fase de construcción concentra la mayor carga operativa del proyecto, mientras que las actividades de documentación, pruebas y despliegue requieren menor cantidad de esfuerzo relativo.

Métricas de Seguimiento y Estimación

Para monitorear el desempeño y avance del proyecto se utilizarán métricas ágiles y de control de calidad.

Sprint Velocity

Permitirá medir la cantidad promedio de Story Points completados por Sprint, ayudando a estimar la capacidad real del equipo y ajustar futuras planeaciones.

Burn-down Chart

Se utilizará para visualizar el trabajo restante respecto al tiempo disponible del Sprint, permitiendo detectar retrasos y desviaciones del cronograma.

Bug Rate

Permitirá medir la cantidad de errores detectados durante las pruebas funcionales y de integración, ayudando a evaluar la estabilidad y calidad del sistema.

Cycle Time

Permitirá medir el tiempo requerido para completar una tarea desde el inicio de desarrollo hasta su finalización, ayudando a identificar cuellos de botella y optimizar el flujo de trabajo.

Plan de Actualización de Estimaciones

Las estimaciones del proyecto serán revisadas y actualizadas al finalizar cada Sprint utilizando información real obtenida durante el desarrollo.

Durante cada revisión se analizarán:

- Historias de usuario completadas.
- Velocity del equipo.
- Esfuerzo invertido.
- Incidencias detectadas.
- Cumplimiento de objetivos.
- Desviaciones del cronograma.
- Resultados de pruebas.

Con base en esta información podrán ajustarse:

- Cronograma del proyecto.
- Estimaciones futuras.
- Esfuerzo restante.
- Prioridades del backlog.
- Distribución de tareas.
- Métricas de desempeño.

Criterios de Éxito de la Estimación

La efectividad del proceso de estimación será evaluada considerando:

- Cumplimiento de objetivos del Sprint.
- Desviación mínima entre esfuerzo estimado y real.
- Estabilidad del Sprint Velocity.
- Reducción de retrasos acumulados.
- Disminución de errores críticos.
- Cumplimiento de tiempos de entrega.

Cuando las estimaciones presenten desviaciones significativas, el equipo realizará ajustes metodológicos durante las Sprint Retrospectives para mejorar la precisión en futuras iteraciones.

Gestión de Datos Históricos

Toda la información relacionada con estimaciones y desempeño será almacenada para generar un historial del proyecto y mejorar futuras planeaciones.

Los datos históricos incluirán:

- Velocity por Sprint.

- Burndown Charts.
- Tiempo real invertido.
- Incidencias registradas.
- Errores detectados.
- Ajustes de cronograma.
- Cambios de alcance.
- Resultados de retrospectivas.

La recopilación de estos datos permitirá mejorar la precisión de estimación y fortalecer procesos futuros de planificación dentro del proyecto.

5.1.2 Plan de personal

Objetivo del Plan de Personal

El presente plan de personal tiene como objetivo definir la estructura del equipo de trabajo, los perfiles requeridos, las habilidades necesarias y las estrategias de formación, integración y motivación utilizadas durante el desarrollo del proyecto. La organización del personal permitirá garantizar una distribución adecuada de responsabilidades y asegurar que el equipo cuente con las competencias técnicas y colaborativas necesarias para cumplir los objetivos del proyecto.

Estructura del Equipo

El equipo de trabajo estará conformado por los siguientes roles:

Rol	Cantidad
Scrum Master	1
Product Owner	1
Desarrolladores	3

La asignación de roles fue definida considerando las necesidades operativas del proyecto y la metodología Scrum utilizada durante el desarrollo.

Descripción de Roles

Scrum Master

Responsable de:

- Facilitar la metodología Scrum.
- Coordinar reuniones Daily Scrum.
- Eliminar bloqueos técnicos y organizacionales.
- Supervisar el flujo de trabajo del Sprint.
- Dar seguimiento a métricas y retrospectivas.

Product Owner

Responsable de:

- Gestionar el Product Backlog.
- Priorizar historias de usuario.
- Validar requerimientos funcionales.
- Mantener comunicación con el responsable del restaurante.
- Aprobar funcionalidades desarrolladas.

Desarrolladores

Responsables de:

- Implementación frontend y backend.
- Integración de módulos.
- Desarrollo de funcionalidades.
- Ejecución de pruebas técnicas.
- Corrección de incidencias.
- Actualización de documentación técnica.

Los desarrolladores trabajarán principalmente sobre módulos funcionales específicos, aunque podrán colaborar en otras áreas según las necesidades del Sprint.

Habilidades Requeridas

El equipo de desarrollo deberá contar con conocimientos intermedios en las siguientes tecnologías y herramientas:

- PHP.
- JavaScript.
- HTML y CSS.
- MySQL.
- Manejo básico de Docker.
- Control de versiones con GitHub.
- Arquitectura MVC.

Asimismo, será necesario contar con habilidades complementarias relacionadas con el trabajo colaborativo y la gestión técnica del proyecto, incluyendo:

- Trabajo en equipo.
- Resolución de problemas.
- Adaptación al cambio.
- Comunicación técnica.
- Documentación de procesos.

- Organización de tareas.
- Capacidad de aprendizaje autónomo.

Estas habilidades fueron consideradas necesarias debido al enfoque colaborativo y ágil utilizado durante el desarrollo del sistema.

Estrategia de Adquisición de Personal

La integración del equipo de trabajo se realizó de forma interna entre los integrantes del grupo académico participante en el proyecto.

La asignación de roles y responsabilidades fue realizada considerando:

- Interés personal.
- Experiencia previa.
- Afinidad técnica.
- Disponibilidad.
- Participación en proyectos anteriores.
- Compatibilidad con las necesidades del sistema.

La estrategia de integración interna fue seleccionada debido a que permite mantener comunicación constante, facilidad de coordinación y conocimiento previo entre los integrantes del equipo.

Plan de Formación Inicial

Durante la fase inicial del proyecto se realizó una revisión de la documentación previa relacionada con el sistema de comanda digital, así como la definición de herramientas, metodologías y tecnologías necesarias para el desarrollo.

Aunque la mayoría de las herramientas ya eran conocidas por el equipo, fue necesaria capacitación adicional en:

- Docker.
- OpenCode.
- Configuración de entornos de desarrollo.
- Automatización de pruebas.
- Integración de contenedores.

La capacitación se realizó mediante:

- Videos tutoriales.
- Documentación oficial.
- Pruebas prácticas.
- Sesiones colaborativas.
- Investigación técnica individual.

- Apoyo entre integrantes del equipo.

El objetivo de esta capacitación fue garantizar uniformidad técnica y reducir problemas relacionados con configuración y compatibilidad del entorno de trabajo.

Estrategia de Colaboración y Soporte Técnico

El proyecto utilizará un enfoque colaborativo de resolución de problemas, permitiendo que cualquier integrante pueda apoyar a otro desarrollador en caso de dificultades técnicas relacionadas con:

- Configuración de herramientas.
- Integración de módulos.
- Problemas de despliegue.
- Errores de dependencias.
- Configuración Docker.
- Uso de GitHub y control de versiones.

Esta estrategia permitirá reducir tiempos muertos y mejorar la continuidad del desarrollo durante los Sprints.

Estrategia de Retención y Motivación

La estrategia de motivación del equipo estará basada principalmente en:

- Cumplimiento de objetivos académicos.
- Desarrollo de competencias profesionales.
- Aprendizaje de herramientas modernas.
- Fortalecimiento de habilidades prácticas.
- Participación colaborativa.
- Experiencia en metodologías ágiles.
- Aplicación de tecnologías DevOps y automatización.

El proyecto busca fomentar un entorno colaborativo orientado al aprendizaje práctico y al desarrollo profesional de los integrantes.

Seguimiento del Desempeño

El desempeño del equipo será monitoreado mediante:

- Cumplimiento de objetivos del Sprint.
- Participación en reuniones Scrum.
- Actualización de tareas en Trello.
- Velocity del equipo.
- Calidad de entregables.

- Participación en pruebas y validaciones.
- Cumplimiento de tiempos estimados.

La información obtenida permitirá identificar áreas de mejora y ajustar la distribución de trabajo cuando sea necesario.

Gestión de Conocimiento

Con el propósito de mantener continuidad operativa y reducir dependencia individual, toda la documentación relevante del proyecto será almacenada y compartida mediante:

- GitHub.
- OneDrive.
- Documentación técnica.
- Manuales de configuración.
- Evidencias de pruebas.
- Reportes de incidencias.

Esto permitirá preservar conocimiento técnico y facilitar futuras actividades de mantenimiento o expansión del sistema.

5.1.3 Recursos

Objetivo del Plan de Recursos

El presente plan de recursos tiene como objetivo identificar, organizar y administrar los recursos necesarios para el desarrollo, pruebas, integración y mantenimiento del sistema de comanda digital.

La correcta administración de recursos permitirá garantizar continuidad operativa, disponibilidad de herramientas y soporte adecuado durante todo el ciclo de vida del proyecto.

Recursos de Hardware

El desarrollo y validación del sistema requerirá la utilización de diferentes dispositivos de cómputo y equipos de prueba.

Los recursos de hardware considerados son:

- Computadoras personales de los integrantes.
- Tablets Android para pruebas operativas.
- Celulares utilizados para validación de interfaces.
- Laptops con Windows para pruebas funcionales.
- Dispositivos móviles complementarios del equipo.

Estos dispositivos permitirán realizar:

- Desarrollo del sistema.
- Pruebas funcionales.
- Validación de interfaces.
- Simulación del entorno operativo real.
- Integración de módulos.
- Pruebas de sincronización en red local.

La utilización de múltiples dispositivos permitirá verificar compatibilidad y estabilidad del sistema en distintos entornos de ejecución.

Recursos de Software

Las herramientas de software utilizadas durante el proyecto serán:

Herramienta	Propósito
Visual Studio Code	Desarrollo del sistema
Docker	Contenerización y estandarización
OpenCode	Validación y pruebas
Trello	Gestión de tareas y seguimiento
MySQL	Gestión de base de datos

Estas herramientas fueron seleccionadas debido a su estabilidad, compatibilidad y disponibilidad gratuita para entornos académicos y de desarrollo colaborativo.

Licencias de Software

Todas las herramientas utilizadas durante el proyecto serán empleadas bajo licencias gratuitas, comunitarias, open source o académicas, evitando costos adicionales y reduciendo riesgos legales relacionados con el uso de software propietario.

La selección de herramientas con licenciamiento libre permitirá:

- Reducir costos operativos.
- Facilitar distribución del entorno.
- Evitar restricciones de instalación.
- Mantener accesibilidad para todos los integrantes.
- Simplificar mantenimiento y actualizaciones.

Equipos de Prueba

Las pruebas funcionales y operativas del sistema serán realizadas utilizando distintos dispositivos para validar comportamiento y compatibilidad.

Los equipos de prueba contemplados incluyen:

- Tablets Android.
- Laptops con Windows.
- Computadoras personales.
- Dispositivos móviles del equipo de desarrollo.

Estos dispositivos permitirán validar:

- Interfaces gráficas.
- Flujo de pedidos.
- Sincronización entre módulos.
- Comunicación en red local.
- Rendimiento operativo.
- Compatibilidad multiplataforma.

Recursos Financieros

El proyecto será desarrollado mediante autofinanciación por parte de los integrantes del equipo.

Debido a que las herramientas utilizadas son gratuitas o de uso académico, los gastos contemplados corresponden principalmente a costos operativos mínimos relacionados con:

- Consumo eléctrico.
- Servicio de internet.
- Transporte.
- Alimentación durante jornadas presenciales.
- Uso de dispositivos personales.

La estrategia de autofinanciación fue seleccionada debido a la naturaleza académica del proyecto y la disponibilidad de herramientas open source.

Recursos de Infraestructura

Para el desarrollo y coordinación del proyecto se dispondrá de los siguientes recursos de infraestructura:

- Una mesa de trabajo por cada dos desarrolladores.
- Acceso a corriente eléctrica.
- Espacios con múltiples conectores eléctricos.
- Conexión Wi-Fi.

- Espacios de trabajo dentro de la universidad.

Aunque gran parte del desarrollo será realizado de forma remota desde casa, cuando sea necesario realizar integración colaborativa o pruebas conjuntas se utilizarán:

- Cubículos.
- Biblioteca.
- Salones disponibles dentro de la universidad.

La utilización de espacios híbridos permitirá mantener flexibilidad operativa y facilitar reuniones técnicas y validaciones integradas.

Sistemas Habilitadores

Conforme a las prácticas establecidas en IEEE 15288 y alineadas con IEEE 16326, se consideran sistemas habilitadores aquellas plataformas y herramientas que permiten soportar las actividades operativas, técnicas y administrativas del proyecto. Los principales sistemas habilitadores utilizados serán:

Sistema	Función
GitHub	Control de versiones y respaldo
Docker	Estandarización del entorno
Trello	Gestión del flujo de trabajo

Estos sistemas permitirán facilitar:

- Integración continua.
- Coordinación del equipo.
- Gestión de cambios.
- Trazabilidad de actividades.
- Administración del proyecto.
- Despliegue e integración de módulos.
- Respaldo de información.

La utilización de estos sistemas busca reducir errores de configuración y mejorar la organización del desarrollo colaborativo.

Gestión de Reservas y Contingencias

Con el propósito de reducir riesgos relacionados con pérdida de información o fallas técnicas, el proyecto implementará mecanismos de respaldo y contingencia. El Scrum Master y el Product Owner serán responsables de mantener respaldos actualizados de:

- Documentación.

- Plantillas.
- Reportes.
- Evidencias.
- Archivos críticos del proyecto.

Las copias de respaldo deberán almacenarse en:

- Dispositivos personales de trabajo.
- Servicios de almacenamiento en nube.
- Repositorios GitHub.
- Unidades físicas de respaldo cuando sea posible.

Asimismo, los desarrolladores deberán:

- Mantener control de versiones mediante GitHub.
- Generar respaldos locales de su trabajo.
- Actualizar periódicamente el repositorio.
- Documentar cambios relevantes.

Estas actividades estarán relacionadas con la gestión de configuración, recuperación de información y continuidad operativa del proyecto.

Seguimiento y Administración de Recursos

El estado y disponibilidad de los recursos será revisado periódicamente durante los Sprints para asegurar:

- Funcionamiento adecuado de herramientas.
- Disponibilidad de infraestructura.
- Compatibilidad de entornos.
- Acceso a repositorios y documentación.
- Correcta sincronización entre dispositivos.
- Disponibilidad de recursos de prueba.

Cualquier incidencia relacionada con recursos deberá registrarse y gestionarse mediante los mecanismos definidos en el proceso de seguimiento y mejora continua del proyecto.

5.1.4 Entrenamiento

Objetivo del Entrenamiento

El plan de entrenamiento tiene como propósito garantizar que los integrantes del proyecto cuenten con los conocimientos técnicos y operativos necesarios para

utilizar correctamente las herramientas, tecnologías y metodologías definidas durante el desarrollo del sistema de comanda digital.

El entrenamiento permitirá reducir errores de configuración, mejorar la integración entre módulos y fortalecer la capacidad técnica del equipo durante las diferentes fases del proyecto.

Asimismo, el proceso de capacitación busca asegurar uniformidad en el uso de herramientas y mantener continuidad operativa durante los Sprints.

Alcance del Entrenamiento

El entrenamiento estará dirigido a todos los integrantes del equipo de desarrollo y cubrirá aspectos relacionados con:

- Configuración del entorno de desarrollo.
- Uso de Docker y contenedores.
- Control de versiones con GitHub.
- Gestión de tareas mediante Trello.
- Configuración de dependencias con Composer y npm.
- Uso de OpenCode para validaciones y pruebas.
- Buenas prácticas de integración y documentación.

La capacitación será aplicada principalmente durante la fase inicial del proyecto y podrá complementarse durante los Sprints cuando se incorporen nuevas herramientas o procesos.

Estrategia de Capacitación

La estrategia de entrenamiento se desarrollará mediante actividades de aprendizaje colaborativo y autoformación técnica.

Los mecanismos de capacitación utilizados serán:

- Revisión de documentación oficial.
- Videos tutoriales.
- Prácticas guiadas.
- Configuración de entornos reales.
- Sesiones colaborativas entre integrantes.
- Resolución conjunta de problemas técnicos.
- Validaciones prácticas mediante pruebas funcionales.

El enfoque práctico permitirá que cada integrante pueda aplicar directamente los conocimientos adquiridos dentro del entorno real del proyecto.

Herramientas y Temas de Capacitación

Herramienta / Tema	Objetivo de Capacitación
Docker	Configuración y manejo de contenedores
GitHub	Control de versiones y trabajo colaborativo
Trello	Gestión de tareas y flujo Kanban
OpenCode	Ejecución de pruebas y validaciones
MySQL	Gestión básica de base de datos
Arquitectura MVC	Organización estructural del sistema

Responsables del Entrenamiento

Responsable	Área de Capacitación
Jesús Vicente Ríos Carrera	Docker y entorno de desarrollo
Jorge Edmundo Osornio Tapia	Gestión de tareas y flujo de trabajo
Guadalupe Yosabeth Pérez Estrada	Seguimiento organizacional y documentación
Oswaldo García Ramírez	OpenCode y validaciones funcionales
Todo el equipo	Apoyo colaborativo y resolución de problemas

Evaluación del Entrenamiento

La efectividad del entrenamiento será validada mediante:

- Configuración correcta de entornos.
- Participación en actividades colaborativas.
- Ejecución exitosa de pruebas.
- Uso adecuado de herramientas.
- Integración correcta con el repositorio.
- Cumplimiento de tareas del Sprint.

En caso de detectarse dificultades técnicas, se programarán sesiones adicionales de apoyo y reforzamiento.

Actualización del Entrenamiento

El plan de capacitación podrá actualizarse durante el proyecto cuando:

- Se incorporen nuevas herramientas.

- Existan cambios tecnológicos relevantes.
- Se detecten deficiencias técnicas.
- Aparezcan nuevos requerimientos operativos.

Las actualizaciones serán registradas como parte del proceso de mejora continua del proyecto.

5.2 Planes de trabajo

Objetivo de los Planes de Trabajo

Los planes de trabajo tienen como propósito definir el alcance funcional, organización de actividades y distribución de módulos que serán desarrollados durante el ciclo de vida del proyecto.

La división del sistema en módulos funcionales permitirá mejorar la organización del desarrollo, facilitar la integración continua y mantener trazabilidad sobre las actividades implementadas durante cada Sprint.

Cada módulo será desarrollado, probado e integrado de manera incremental conforme a la planificación definida en el Product Backlog y los objetivos establecidos para cada Sprint.

Alcance del Trabajo

El alcance del proyecto contempla el desarrollo de un sistema de comanda digital para restaurante, compuesto por módulos funcionales interconectados que permitirán automatizar procesos operativos, administrativos y de atención al cliente.

El sistema estará orientado a mejorar:

- Gestión de pedidos.
- Control operativo.
- Comunicación entre áreas.
- Sincronización en tiempo real.
- Administración de usuarios y productos.
- Flujo de trabajo del restaurante.

Implementación de Gestión de Mesas

El módulo de gestión de mesas permitirá la asignación, control y administración de mesas disponibles dentro del restaurante.

Cada mesa podrá asociarse a una tableta utilizada para la generación de pedidos y seguimiento de órdenes.

Las principales funciones del módulo incluyen:

- Asignación de mesas.

- Liberación de mesas al finalizar el servicio.
- Visualización de estado de ocupación.
- Prevención de asignaciones duplicadas.
- Sincronización con pedidos activos.

La implementación de este módulo busca mejorar la organización operativa y evitar conflictos relacionados con pedidos simultáneos o asignaciones incorrectas.

Implementación del Menú Digital

El módulo de menú digital permitirá la visualización dinámica de productos disponibles para la generación de pedidos desde las tabletas del restaurante.

El sistema permitirá mostrar:

- Categorías de productos.
- Platos disponibles.
- Precios actualizados.
- Disponibilidad de productos.
- Descripciones básicas de alimentos.

La actualización dinámica del menú permitirá mantener información sincronizada entre cocina, caja y dispositivos de atención.

Implementación del Módulo de Meseros

El módulo de meseros permitirá gestionar pedidos activos y coordinar el flujo operativo relacionado con atención al cliente.

Entre sus funciones principales se encuentran:

- Visualización de pedidos pagados.
- Consulta de órdenes activas.
- Modificación de prioridades de producción.
- Liberación de pedidos completados.
- Seguimiento de estados de entrega.

La implementación de este módulo permitirá optimizar la comunicación entre cocina y atención al cliente.

Implementación del Módulo de Cocina

El módulo de cocina permitirá visualizar y administrar pedidos en preparación dentro del restaurante.

Las funcionalidades consideradas incluyen:

- Visualización de pedidos por orden de llegada.
- Priorización de órdenes.
- Actualización de estado de preparación.
- Liberación de pedidos finalizados.
- Sincronización automática con caja y meseros.

Este módulo permitirá mejorar la organización del flujo de producción y reducir tiempos de espera.

Implementación del Módulo de Caja

El módulo de caja permitirá realizar operaciones relacionadas con pagos y control financiero básico del restaurante.

Las funciones principales incluyen:

- Cobro de pedidos.
- Generación de tickets.
- Consulta de cuentas activas.
- Registro de pagos.
- Cierre de órdenes.
- Validación de estados de pedidos.

La integración con los demás módulos permitirá mantener sincronización en tiempo real del estado operativo del sistema.

Implementación del Panel de Administración

El panel administrativo permitirá gestionar configuraciones generales y operaciones administrativas del sistema.

Las funcionalidades consideradas incluyen:

- Gestión de usuarios.
- Administración de inventario.
- Gestión de productos y categorías.
- Configuración general del sistema.
- Generación de reportes administrativos.
- Control de accesos y permisos.

La separación de este módulo permitirá centralizar actividades administrativas y mejorar el control operativo del sistema.

Organización del Trabajo por Sprint

Las funcionalidades del sistema serán distribuidas progresivamente durante los diferentes Sprints definidos para el proyecto.

La planificación de trabajo considerará:

- Prioridad funcional.
- Dependencias entre módulos.
- Complejidad técnica.
- Tiempo estimado.
- Riesgos de integración.
- Disponibilidad del equipo.

Cada Sprint incluirá:

- Objetivos específicos.
- Historias de usuario asociadas.
- Actividades técnicas.
- Validaciones funcionales.
- Integración incremental.

Seguimiento del Trabajo

El seguimiento de actividades será realizado mediante:

- Tablero Kanban en Trello.
- Daily Scrum.
- Sprint Reviews.
- Sprint Retrospectives.
- Métricas ágiles del proyecto.

El avance de cada módulo será monitoreado mediante:

- Velocity del Sprint.
- Burn-down Chart.
- Cumplimiento de historias de usuario.
- Estado de tareas.
- Registro de incidencias.

Validación del Trabajo

Cada módulo implementado deberá cumplir con:

- Requisitos funcionales definidos.
- Integración correcta con otros componentes.
- Validaciones funcionales aprobadas.
- Pruebas operativas satisfactorias.
- Revisión técnica del equipo.
- Cumplimiento de la Definition of Done (DoD).

Únicamente después de cumplir estas condiciones las funcionalidades podrán considerarse finalizadas dentro del Sprint correspondiente.

5.2.2 Actividades (WBS)

Objetivo de la WBS

La Estructura del Desglose del Trabajo (Work Breakdown Structure — WBS) tiene como propósito organizar y dividir el proyecto en componentes, entregables y actividades más pequeñas y controlables, permitiendo mejorar la planificación, seguimiento y administración del trabajo durante el ciclo de vida del sistema.

La WBS facilita:

- Organización del proyecto.
- Asignación de responsabilidades.
- Seguimiento del avance.
- Estimación de esfuerzo.
- Control de entregables.
- Trazabilidad de actividades.
- Gestión de dependencias entre módulos.

La estructura definida se encuentra alineada con las actividades y módulos funcionales contemplados dentro del alcance del proyecto.

Nivel 1 — Proyecto General

1. Sistema de Comanda Digital para Restaurante

Proyecto principal orientado al desarrollo de un sistema digital para automatizar procesos operativos y administrativos del restaurante.

Nivel 2 — Gestión y Planeación

1.1 Planeación del Proyecto

1.1.1 Definición de requerimientos

- Identificación de necesidades del restaurante.
- Levantamiento de requisitos funcionales.
- Identificación de restricciones y supuestos.

1.1.2 Organización del proyecto

- Asignación de roles.
- Definición de responsabilidades.
- Organización de Sprints.

1.1.3 Estimación y cronograma

- Estimación de esfuerzo.
- Definición de Story Points.
- Planeación de Sprints.
- Elaboración del cronograma.

1.1.4 Gestión documental

- Elaboración de documentación IEEE 16326.
- Actualización de reportes.
- Generación de evidencias.

Nivel 2 — Configuración de Infraestructura

1.2 Configuración del Entorno

1.2.1 Configuración de Docker

- Creación de contenedores.
- Configuración de imágenes.
- Integración de servicios.

1.2.2 Configuración del repositorio GitHub

- Creación de ramas.
- Configuración de control de versiones.
- Gestión de merges.

1.2.3 Configuración de base de datos

- Instalación de MySQL.
- Diseño inicial de tablas.
- Configuración de conexiones.

1.2.4 Configuración de herramientas

- Instalación de Visual Studio Code.
- Configuración Composer y npm.
- Configuración OpenCode.

Nivel 2 — Desarrollo del Sistema

1.3 Desarrollo de Funcionalidades

1.3.1 Implementación del módulo de autenticación

Actividades

- Diseño de login.
- Gestión de roles.

- Validación de usuarios.
- Control de sesiones.
- Restricción de accesos.

1.3.2 Implementación de gestión de mesas

Actividades

- Registro de mesas.
- Asignación de tablets.
- Liberación de mesas.
- Validación de disponibilidad.
- Sincronización de estados.

1.3.3 Implementación del menú digital

Actividades

- Gestión de categorías.
- Visualización de productos.
- Configuración de precios.
- Validación de disponibilidad.
- Actualización dinámica del menú.

1.3.4 Implementación del módulo de meseros

Actividades

- Visualización de pedidos.
- Consulta de órdenes activas.
- Liberación de pedidos.
- Gestión de prioridades.
- Actualización de estados.

1.3.5 Implementación del módulo de cocina

Actividades

- Visualización de órdenes.
- Priorización de pedidos.
- Actualización de preparación.
- Liberación de órdenes completadas.
- Sincronización con caja y meseros.

1.3.6 Implementación del módulo de caja

Actividades

- Cobro de pedidos.
- Registro de pagos.

- Generación de tickets.
- Consulta de cuentas.
- Validación de órdenes.

1.3.7 Implementación del panel administrativo

Actividades

- Gestión de usuarios.
- Administración de inventario.
- Configuración del sistema.
- Gestión de productos.
- Generación de reportes.

Nivel 2 — Integración y Pruebas

1.4 Validación del Sistema

1.4.1 Pruebas funcionales

- Validación de módulos.
- Pruebas de formularios.
- Validación de flujos operativos.

1.4.2 Pruebas de integración

- Comunicación entre módulos.
- Sincronización de pedidos.
- Validación de red local.

1.4.3 Pruebas operativas

- Simulación del restaurante.
- Validación de tiempos de respuesta.
- Pruebas en tablets y laptops.

1.4.4 Corrección de incidencias

- Registro de errores.
- Ajustes de funcionalidades.
- Validación posterior.

Nivel 2 — Gestión del Proyecto Scrum

1.5 Seguimiento y Control

1.5.1 Daily Scrum

- Seguimiento diario.
- Identificación de bloqueos.
- Coordinación del Sprint.

1.5.2 Sprint Planning

- Planeación de actividades.
- Selección de historias de usuario.
- Asignación de tareas.

1.5.3 Sprint Review

- Validación de entregables.
- Revisión funcional.
- Retroalimentación del cliente.

1.5.4 Sprint Retrospective

- Identificación de mejoras.
- Revisión de problemas.
- Actualización de procesos.

1.5.5 Seguimiento de métricas

- Velocity.
- Burn-down Chart.
- Bug Rate.
- Cycle Time.

Nivel 2 — Despliegue y Cierre

1.6 Liberación del Sistema

1.6.1 Preparación de despliegue

- Configuración final.
- Verificación de servicios.
- Validación de contenedores.

1.6.2 Capacitación de usuarios

- Explicación de funcionamiento.
- Validación de operación.
- Entrega de manuales.

1.6.3 Aceptación final

- Validación del cliente.
- Firma de conformidad.
- Registro de evidencias.

1.6.4 Cierre del proyecto

- Respaldo de documentación.
- Organización de entregables.

- Cierre administrativo del proyecto.

5.2.3 Cronograma y presupuesto

Asignación del Cronograma

Objetivo del Cronograma

El cronograma del proyecto tiene como propósito organizar temporalmente las actividades necesarias para el desarrollo, integración, pruebas y liberación del sistema de comanda digital.

La planificación temporal se realizará utilizando iteraciones ágiles mediante Sprints, permitiendo seguimiento continuo, control del avance y adaptación progresiva conforme evolucionen los requerimientos y resultados del proyecto.

El cronograma se encuentra alineado con las actividades definidas dentro de la Estructura de Desglose del Trabajo (WBS).

Estrategia de Planificación Temporal

El proyecto será desarrollado mediante Sprints iterativos de 11 días hábiles distribuidos entre:

- Planeación.
- Desarrollo.
- Integración.
- Validación.
- Documentación.
- Corrección de incidencias.

El seguimiento del avance será realizado utilizando:

- Tablero Kanban en Trello.
- Daily Scrum.
- Sprint Reviews.
- Sprint Retrospectives.
- Métricas ágiles del proyecto.

La planeación iterativa permitirá ajustar actividades y estimaciones conforme avance el desarrollo del sistema.

Relaciones de Programación entre Actividades

Las actividades del proyecto presentan dependencias funcionales y técnicas que determinan su secuencia de ejecución.

Actividades Secuenciales

Las siguientes actividades requieren completarse antes de iniciar etapas posteriores:

Actividad Dependiente	Requiere
Implementación de módulos	Configuración del entorno
Integración de pedidos	Implementación de autenticación
Pruebas integrales	Finalización de módulos principales
Validación operativa	Integración completa del sistema
Despliegue final	Aprobación de pruebas

Actividades Paralelas

Algunas actividades podrán desarrollarse simultáneamente para optimizar tiempos de ejecución:

Actividades Paralelas

- Documentación y desarrollo
- Configuración de entorno y planeación
- Desarrollo frontend y backend
- Pruebas unitarias y desarrollo incremental
- Validaciones funcionales y corrección de incidencias

La ejecución paralela permitirá mejorar la productividad y reducir tiempos muertos durante los Sprints.

Restricciones del Cronograma

El cronograma considera diversas restricciones externas que pueden afectar las fechas estimadas del proyecto.

Las principales restricciones identificadas son:

- Disponibilidad parcial del equipo por carga académica.
- Dependencia de horarios escolares.
- Disponibilidad de espacios dentro de la universidad.
- Disponibilidad de dispositivos de prueba.
- Dependencia de red local para validaciones integradas.
- Posibles problemas de compatibilidad técnica.
- Limitaciones de tiempo para pruebas presenciales.

Estas restricciones fueron consideradas durante la estimación de tiempo y definición de márgenes de contingencia.

Cronograma General del Proyecto

El proyecto se desarrollará aproximadamente durante 14 semanas de trabajo activo.

Sprint	Actividades Principales	Duración
Sprint 1	Planeación, documentación y configuración del entorno	10 días
Sprint 2	Autenticación, gestión de mesas y pedidos	10 días
Sprint 3	Integración de cocina y meseros	7 días
Sprint 4	Implementación de caja	10 días
Sprint 5	Integración con inventarios y panel administrativo	10 días
Sprint 6	Validaciones funcionales y pruebas integradas	10 días
Sprint 7	Corrección de incidencias, documentación y cierre	10 días

Hitos del Proyecto

Los hitos representan eventos importantes que permitirán medir el avance real del proyecto mediante resultados verificables.

Hito 1 — Configuración Inicial del Proyecto

Sprint 1

Incluye:

- Revisión de documentación heredada.
- Configuración Docker.
- Configuración GitHub.
- Definición del Product Backlog.
- Organización inicial del proyecto.

Indicadores de cumplimiento

- Entorno funcional configurado.
- Acceso correcto al repositorio.
- Backlog aprobado.

Hito 2 — Implementación Base del Sistema

Sprint 2

Incluye:

- Sistema de autenticación.
- Gestión de mesas.
- Generación inicial de pedidos.

Indicadores de cumplimiento

- Login funcional.
- Asignación correcta de mesas.
- Flujo inicial de pedidos operativo.

Hito 3 — Integración Operativa del Restaurante

Sprint 3

Incluye:

- Integración de cocina.
- Integración de meseros.
- Gestión de pedidos pagados.

Indicadores de cumplimiento

- Pedidos sincronizados correctamente.
- Visualización funcional en cocina y meseros.
- Actualización correcta de estados.

Hito 4 — Implementación Financiera y Administrativa

Sprint 4 y 5

Incluye:

- Módulo de caja.
- Generación de tickets.
- Integración con inventarios.
- Panel administrativo.

Indicadores de cumplimiento

- Cobros funcionales.
- Inventario actualizado automáticamente.
- Gestión administrativa operativa.

Hito 5 — Validación Final y Liberación

Sprint 6 y 7

Incluye:

- Pruebas funcionales.
- Integración completa.
- Corrección de errores.
- Documentación final.

Indicadores de cumplimiento

- Flujo completo validado.
- Errores críticos corregidos.
- Aceptación funcional aprobada.

Técnicas de Representación del Cronograma

Para la visualización y seguimiento del proyecto se utilizarán:

- Diagramas de Gantt.
- Tableros Kanban.
- Burndown Charts.
- Seguimiento mediante Sprints.
- Métricas de Velocity.

Estas herramientas permitirán monitorear el estado del proyecto y detectar desviaciones respecto al cronograma planeado.

Asignación del Presupuesto

Objetivo del Presupuesto

El presupuesto del proyecto tiene como propósito estimar y controlar los recursos económicos necesarios para el desarrollo del sistema de comanda digital. Debido a la naturaleza académica del proyecto, el presupuesto será estimado de forma referencial considerando recursos autofinanciados y herramientas gratuitas.

Presupuesto por Categoría

Recursos Humanos

El principal costo del proyecto corresponde al esfuerzo técnico invertido por el equipo de trabajo.

Actividad	Recurso	Horas Estimadas
-----------	---------	-----------------

Planeación y documentación	Scrum Master / Product Owner	210 horas 210 horas
Desarrollo de módulos	Equipo de desarrollo	336 horas
Arquitectura y diseño	Equipo técnico	126 horas
Pruebas y validación	Equipo completo	126
Despliegue y cierre	Equipo completo	42 horas

Infraestructura

Los costos de infraestructura consideran principalmente recursos operativos utilizados durante el desarrollo.

Recurso	Descripción
Energía eléctrica	Consumo durante jornadas de trabajo
Internet	Desarrollo remoto y sincronización
Equipos personales	Computadoras y laptops
Tablets Android	Validaciones funcionales
Red Wi-Fi	Simulación operativa del restaurante

Software y Herramientas

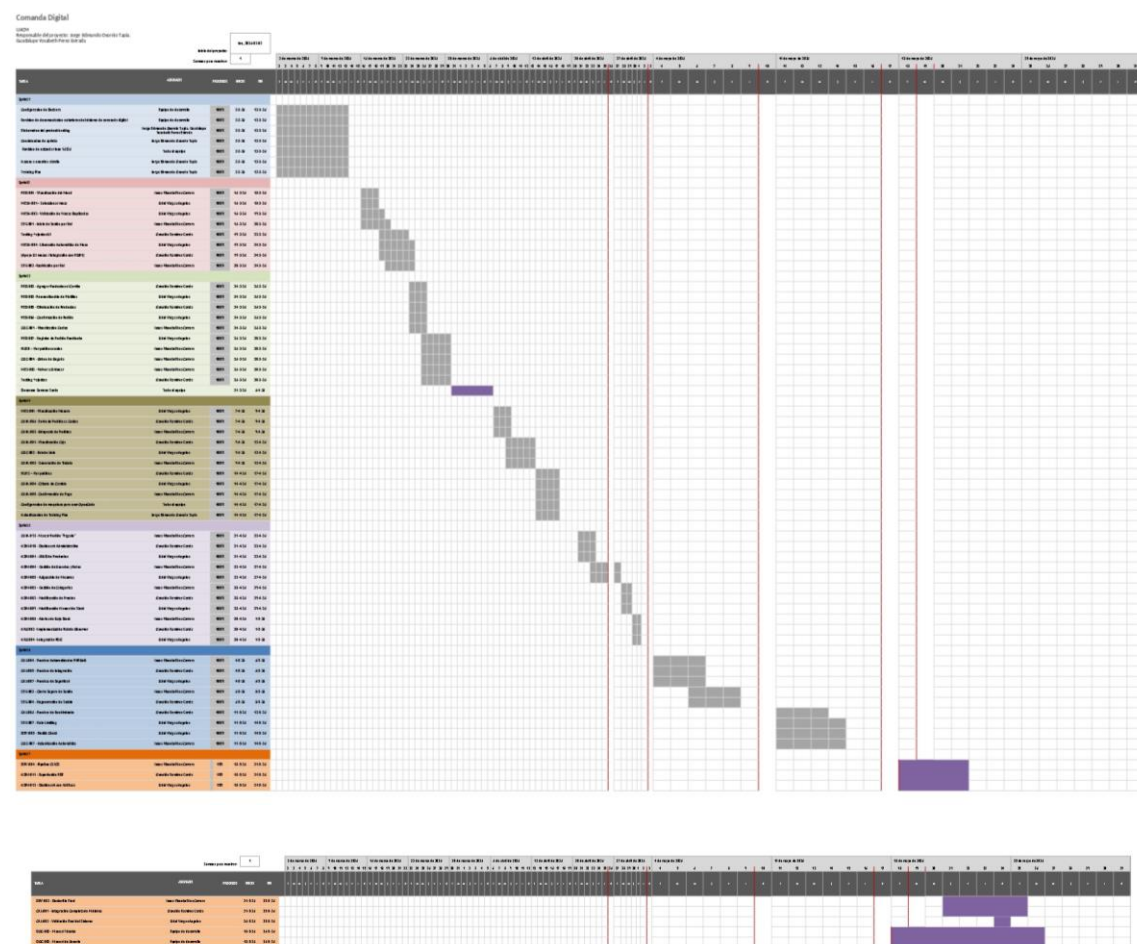
El proyecto utilizará herramientas gratuitas o comunitarias, por lo que no se contemplan costos de licenciamiento comercial.

Herramienta	Tipo de Licencia
Docker Community Edition	Gratuita
GitHub	Gratuita académica
Trello	Gratuita
Visual Studio Code	Gratuita
MySQL Community	Gratuita
OpenCode	Gratuita

Reserva de Contingencia

Se mantendrá una reserva de contingencia destinada a atender posibles incidencias relacionadas con:

- Problemas de configuración Docker.
- Errores de integración.
- Fallas de sincronización.
- Problemas de autenticación.
- Correcciones críticas.
- Retrasos operativos.



6. Evaluación y control

6.1 Gestión de requisitos

Objetivo de Evaluación y Control

El proceso de evaluación y control tiene como propósito supervisar el cumplimiento de los objetivos del proyecto, validar el avance de las actividades planificadas y garantizar que los entregables desarrollados cumplan con los requisitos funcionales, técnicos y de calidad definidos para el sistema.

Las actividades de evaluación y control permitirán identificar desviaciones respecto al cronograma, alcance, esfuerzo y calidad del proyecto, facilitando la aplicación de acciones correctivas durante el desarrollo.

Este proceso se encuentra alineado con las prácticas de seguimiento y control establecidas en IEEE 16326.

Gestión de Requisitos

Obtención de Requisitos

Durante la etapa inicial del proyecto se definieron los requisitos funcionales y no funcionales del sistema mediante:

- Entrevistas con el administrador del restaurante.
- Entrevistas con meseros.
- Observación directa del flujo operativo.
- Análisis de procesos internos de cocina y caja.

La información recopilada permitió identificar necesidades operativas, restricciones y problemas relacionados con la gestión manual de pedidos dentro del restaurante.

La versión aprobada de dichos requisitos quedó documentada dentro de la Especificación de Requisitos de Software (SRS), utilizada como línea base funcional del proyecto.

Transformación de Requisitos a Historias de Usuario

Los requisitos funcionales definidos en la SRS fueron transformados en Historias de Usuario organizadas dentro del Product Backlog para facilitar su implementación mediante metodología Scrum.

Cada Historia de Usuario incluye:

- Identificador único.

- Prioridad.
- Estado.
- Valor de negocio.
- Story Points.
- Responsable asignado.
- Criterios de aceptación.

La utilización de Historias de Usuario permite mejorar la organización incremental del desarrollo y facilitar el seguimiento de funcionalidades implementadas durante cada Sprint.

Trazabilidad de Requisitos

Cada Historia de Usuario mantiene relación directa con uno o varios requisitos funcionales definidos en la SRS, permitiendo conservar trazabilidad entre:

- Requisitos.
- Historias de Usuario.
- Actividades del Sprint.
- Módulos implementados.
- Casos de prueba.
- Resultados de validación.

La trazabilidad permitirá:

- Identificar el estado de implementación de cada requisito.
- Verificar cobertura funcional.
- Facilitar validaciones.
- Gestionar cambios de alcance.
- Mantener control documental del proyecto.

Gestión de Cambios

Cualquier modificación relacionada con requisitos funcionales, alcance o comportamiento operativo del sistema deberá ser evaluada antes de su implementación.

El análisis de cambios considerará:

- Impacto en alcance.
- Impacto en cronograma.
- Impacto en presupuesto.
- Impacto técnico.
- Riesgos asociados.
- Dependencias funcionales.
- Impacto sobre módulos integrados.

Los cambios serán discutidos y evaluados durante reuniones periódicas con stakeholders y responsables del proyecto.

Únicamente después de su aprobación los cambios podrán incorporarse al Product Backlog y planificarse dentro de futuros Sprints.

Seguimiento y Control del Proyecto

El seguimiento del proyecto será realizado de forma continua mediante prácticas ágiles de supervisión y control.

Las principales actividades de seguimiento incluyen:

- Daily Scrum.
- Sprint Planning.
- Sprint Review.
- Sprint Retrospective.
- Revisión de métricas ágiles.
- Validación de entregables.

Estas actividades permitirán monitorear:

- Avance del Sprint.
- Cumplimiento de objetivos.
- Estado de tareas.
- Riesgos identificados.
- Incidencias técnicas.
- Cumplimiento de cronograma.

Métricas de Evaluación

Para evaluar el desempeño y progreso del proyecto se utilizarán las siguientes métricas:

Sprint Velocity

Permitirá medir la cantidad de Story Points completados en cada Sprint y evaluar la capacidad real del equipo.

Burn-down Chart

Permitirá visualizar el trabajo restante respecto al tiempo disponible del Sprint y detectar posibles retrasos.

Bug Rate

Permitirá medir la cantidad de errores detectados durante pruebas funcionales e integración.

Cycle Time

Permitirá medir el tiempo promedio requerido para completar tareas individuales desde el inicio hasta su finalización.

Evaluación de Calidad

La calidad del sistema será evaluada mediante:

- Pruebas funcionales.
- Pruebas de integración.
- Validaciones operativas.
- Revisión de código.
- Verificación de criterios de aceptación.
- Cumplimiento de la Definition of Done (DoD).

Cada funcionalidad deberá cumplir los criterios establecidos antes de considerarse finalizada dentro del Sprint correspondiente.

Gestión de Incidencias

Todas las incidencias detectadas durante el desarrollo o pruebas serán registradas y clasificadas según:

- Nivel de severidad.
- Impacto operativo.
- Prioridad.
- Módulo afectado.
- Estado de resolución.

Las incidencias serán analizadas durante reuniones de seguimiento para definir acciones correctivas y prevenir recurrencia.

Control del Cronograma y Presupuesto

El control del cronograma y presupuesto será realizado mediante comparación periódica entre:

- Horas estimadas vs horas reales.
- Actividades planificadas vs completadas.
- Recursos asignados vs utilizados.

- Consumo de reservas de contingencia.

Las desviaciones detectadas permitirán realizar ajustes sobre:

- Planeación de Sprints.
- Distribución de tareas.
- Estimaciones futuras.
- Prioridades del Product Backlog.

Gestión de Evidencias y Reportes

Toda la información relacionada con evaluación y control será documentada mediante:

- Reportes de Sprint.
- Historial de incidencias.
- Evidencias de pruebas.
- Métricas ágiles.
- Registros de retrospectivas.
- Validaciones funcionales.
- Reportes de avance.

La documentación permitirá mantener trazabilidad y respaldo del estado del proyecto durante todo el ciclo de desarrollo.

Mejora Continua

Los resultados obtenidos durante las actividades de evaluación y control serán utilizados para implementar mejoras continuas sobre:

- Procesos de desarrollo.
- Planeación de actividades.
- Estimaciones.
- Validaciones.
- Integración de módulos.
- Gestión de riesgos.

Las acciones de mejora serán revisadas periódicamente durante las Sprint Retrospectives para verificar su efectividad y actualizar los procesos del proyecto cuando sea necesario.

6.2 Control de alcance Objetivo del Control de Alcance

El control de alcance tiene como propósito garantizar que el proyecto desarrolle únicamente las funcionalidades, actividades y entregables previamente aprobados dentro de la línea base del proyecto.

El proceso de control permitirá evitar crecimiento descontrolado del alcance (scope creep), reducir desviaciones del cronograma y mantener estabilidad sobre los recursos, costos y actividades planificadas.

Todas las modificaciones deberán seguir un proceso formal de evaluación, aprobación y actualización documental.

Línea Base del Alcance

La línea base del alcance estará conformada por:

- Especificación de Requisitos de Software (SRS).
- Product Backlog aprobado.
- Historias de Usuario.
- Estructura de Desglose del Trabajo (WBS).
- Cronograma aprobado.
- Entregables definidos para cada Sprint.

Cualquier actividad o funcionalidad no contemplada dentro de estos elementos será considerada fuera del alcance inicial del proyecto.

Proceso de Control de Cambios

Las solicitudes de cambio relacionadas con requisitos, funcionalidades, entregables o actividades deberán ser gestionadas mediante un proceso formal de evaluación y aprobación.

El proceso de control de cambios incluirá las siguientes etapas:

1. Solicitud del cambio

La solicitud deberá realizarse formalmente mediante reunión con los responsables del proyecto y posteriormente registrarse documentalmente.

La solicitud deberá incluir:

- Descripción del cambio.
- Justificación.
- Objetivo del cambio.
- Área afectada.
- Prioridad.
- Responsable solicitante.

2. Análisis técnico y operativo

El equipo realizará un análisis para determinar:

- Impacto funcional.
- Impacto técnico.
- Dependencias afectadas.
- Riesgos asociados.
- Compatibilidad con módulos existentes.
- Complejidad de implementación.

3. Evaluación de impacto

Cada cambio será evaluado considerando:

- Impacto en alcance.
- Impacto en cronograma.
- Impacto en esfuerzo técnico.
- Impacto en presupuesto.
- Impacto en calidad.
- Impacto en integración y pruebas.

4. Revisión y aprobación

Las solicitudes serán discutidas durante:

- Sprint Review.
- Reuniones quincenales.
- Reuniones de seguimiento con stakeholders.

Únicamente después de la aprobación formal el cambio podrá integrarse al Product Backlog y planificarse para futuros Sprints.

5. Actualización documental

Cuando un cambio sea aprobado deberán actualizarse:

- SRS.
- Historias de Usuario.
- Product Backlog.
- WBS.
- Cronograma.
- Documentación técnica relacionada.

Esto permitirá mantener trazabilidad y consistencia documental del proyecto.

Comité de Control de Cambios

Las decisiones relacionadas con cambios de alcance serán evaluadas principalmente por:

Rol	Responsabilidad
Product Owner	Validación funcional y prioridad
Scrum Master	Evaluación operativa y seguimiento
Equipo de desarrollo	Evaluación técnica
Dueño o administrador del restaurante	Validación operativa del negocio

Cuando el cambio afecte directamente la operación real del restaurante o modifique procesos críticos del sistema, será necesaria la validación del responsable del restaurante antes de su aprobación final.

Gestión de Prioridades

El Product Backlog será priorizado continuamente para asegurar que las funcionalidades más importantes sean desarrolladas primero.

La priorización considerará:

- Valor de negocio.
- Riesgo técnico.
- Dependencias funcionales.
- Complejidad de implementación.
- Impacto operativo.

Solo podrán desarrollarse tareas y funcionalidades previamente aprobadas dentro de la planificación oficial del proyecto.

Control de Scope Creep

Con el propósito de evitar crecimiento descontrolado del proyecto:

- No se desarrollarán funcionalidades fuera de la WBS aprobada.
- Los cambios no autorizados serán rechazados.
- Toda modificación deberá pasar por análisis formal.
- Se evitarán cambios innecesarios durante Sprints activos.
- Las nuevas solicitudes serán priorizadas para futuras iteraciones.

La aplicación de estas restricciones permitirá mantener estabilidad sobre:

- Cronograma.
- Presupuesto.
- Recursos.
- Calidad.
- Capacidad del equipo.

Seguimiento del Alcance

El control del alcance será monitoreado mediante:

- Revisión del Product Backlog.
- Seguimiento de Historias de Usuario.
- Validación de entregables.
- Sprint Reviews.
- Métricas de avance.
- Comparación entre trabajo planificado y trabajo ejecutado.

Las desviaciones identificadas permitirán aplicar acciones correctivas oportunas y actualizar la planeación del proyecto cuando sea necesario.

Evidencia y Trazabilidad

Todas las solicitudes de cambio y decisiones relacionadas con el alcance deberán quedar respaldadas mediante:

- Actas de reunión.
- Actualizaciones del Product Backlog.
- Historial de cambios.
- Reportes de Sprint.
- Evidencias en Trello y GitHub.
- Actualizaciones documentales.

Esto permitirá conservar trazabilidad completa sobre la evolución del alcance durante todo el ciclo de vida del proyecto.

6.3 Control de cronograma

Objetivo del Control del Cronograma

El control del cronograma tiene como propósito supervisar el avance real del proyecto respecto a la planificación establecida, permitiendo identificar desviaciones de tiempo, retrasos y riesgos que puedan afectar el cumplimiento de los objetivos definidos para cada Sprint y para el proyecto en general.

El seguimiento continuo del cronograma permitirá implementar acciones correctivas oportunas y mantener estabilidad en la ejecución de las actividades definidas dentro de la WBS.

Línea Base del Cronograma

La línea base del cronograma estará conformada por:

- Cronograma general aprobado.
- Planeación de Sprints.
- Hitos del proyecto.
- Actividades de la WBS.
- Historias de Usuario comprometidas por Sprint.
- Estimaciones de esfuerzo y duración.

Toda modificación al cronograma deberá seguir el proceso formal de control de cambios definido para el proyecto.

Mecanismos de Seguimiento del Cronograma

El avance del proyecto será supervisado mediante:

- Daily Scrum.
- Sprint Reviews.
- Sprint Retrospectives.
- Tablero Kanban en Trello.
- Burn-down Charts.
- Seguimiento de Story Points completados.
- Comparación entre trabajo planeado y trabajo ejecutado.

Estas actividades permitirán monitorear el estado real del proyecto y detectar posibles retrasos de manera temprana.

Seguimiento de Hitos

Los hitos definidos dentro del cronograma serán utilizados como puntos de control para evaluar el avance real del proyecto.

Cada hito será evaluado considerando:

- Funcionalidades completadas.
- Historias de Usuario terminadas.
- Integraciones realizadas.
- Resultados de pruebas.

- Cumplimiento de Definition of Done (DoD).
- Evidencias funcionales generadas.

El cumplimiento de hitos permitirá validar el progreso incremental del sistema durante los Sprints.

Métricas de Control del Cronograma

Para supervisar el desempeño temporal del proyecto se utilizarán las siguientes métricas:

Sprint Velocity

Permitirá medir la cantidad promedio de Story Points completados por Sprint para evaluar la capacidad real del equipo.

Burn-down Chart

Permitirá visualizar el trabajo restante respecto al tiempo disponible del Sprint y detectar posibles desviaciones del cronograma.

Cycle Time

Permitirá medir el tiempo requerido para completar tareas individuales y detectar cuellos de botella operativos.

Técnica de Valor Ganado (Earned Value Management)

Con el propósito de evaluar el desempeño temporal del proyecto, se utilizarán conceptos básicos de la técnica de Valor Ganado (Earned Value Management — EVM).

La técnica permitirá comparar:

- Trabajo planeado.
- Trabajo ejecutado.
- Avance real del proyecto.

Planned Value (PV)

Representa el valor del trabajo planificado hasta una fecha determinada.

Earned Value (EV)

Representa el valor del trabajo realmente completado respecto a lo planeado.

Schedule Variance (SV)

Permitirá medir la desviación del cronograma mediante:

$$SV=EV-PV$$

Interpretación:

- $SV > 0 \rightarrow$ Proyecto adelantado.
- $SV = 0 \rightarrow$ Proyecto conforme al plan.
- $SV < 0 \rightarrow$ Proyecto retrasado.

Schedule Performance Index (SPI)

Permitirá medir la eficiencia del cronograma mediante:

$$SPI=EV|PV$$

Interpretación:

- $SPI > 1 \rightarrow$ Avance superior al planeado.
- $SPI = 1 \rightarrow$ Avance conforme al cronograma.
- $SPI < 1 \rightarrow$ Retraso respecto a la planeación.

Acciones Correctivas del Cronograma

Cuando se detecten desviaciones importantes del cronograma podrán implementarse acciones como:

- Redistribución de tareas.
- Repriorización del Product Backlog.
- Ajuste de carga de trabajo.
- Replanificación de actividades.
- Incremento de validaciones parciales.
- División de tareas complejas.

- Reasignación temporal de responsables.

Las acciones correctivas serán evaluadas durante Sprint Reviews y Sprint Retrospectives.

Frecuencia de Evaluación

El control del cronograma será realizado mediante:

Actividad	Frecuencia
Daily Scrum	Diario
Revisión de Trello	Continua
Burn-down Chart	Diario
Sprint Review	Al finalizar cada Sprint
Evaluación SPI y SV	Dependiendo el avance del proyecto
Revisión de hitos	Según cronograma

La revisión continua permitirá detectar desviaciones tempranamente y mantener estabilidad sobre la planificación general del proyecto.

Gestión de Evidencias del Cronograma

Toda la información relacionada con el seguimiento del cronograma será documentada mediante:

- Reportes de Sprint.
- Burn-down Charts.
- Historial de Story Points.
- Registro de incidencias.
- Evidencias de Trello.
- Resultados de retrospectivas.
- Métricas SPI y SV.

La documentación permitirá mantener trazabilidad y respaldo histórico del desempeño temporal del proyecto.

Control del Presupuesto

Objetivo del Control Presupuestal

El control presupuestal tiene como propósito supervisar el uso de recursos y el esfuerzo invertido durante el desarrollo del proyecto, permitiendo comparar el trabajo ejecutado respecto a la planeación inicial.

Debido a la naturaleza académica del proyecto, el presupuesto se basa principalmente en el control del esfuerzo técnico y uso de recursos operativos.

Línea Base del Presupuesto

La línea base presupuestal estará conformada por:

- Estimación total de horas-hombre.
- Distribución de esfuerzo por actividad.
- Recursos asignados.
- Reservas de contingencia.
- Cronograma aprobado.

Toda modificación presupuestal deberá ser evaluada mediante el proceso de control de cambios.

Seguimiento del Presupuesto

El seguimiento presupuestal será realizado mediante:

- Comparación de horas estimadas vs reales.
- Seguimiento de actividades completadas.
- Registro de incidencias.
- Revisión de consumo de recursos.
- Seguimiento del esfuerzo técnico por Sprint.

Métricas de Control Presupuestal

Actual Cost (AC)

Representa el esfuerzo o costo real invertido durante el desarrollo de las actividades.

Cost Variance (CV)

Permitirá medir la desviación entre el valor ganado y el costo real:

$$CV = EV - ACCV = EV - AC$$

Interpretación:

- $CV > 0$ → Menor costo al estimado.
- $CV = 0$ → Costo conforme a la planeación.
- $CV < 0$ → Sobrecosto.

Cost Performance Index (CPI)

Permitirá medir la eficiencia del uso de recursos:

$$CPI = EV / AC$$

Interpretación:

- $CPI > 1$ → Uso eficiente de recursos.
- $CPI = 1$ → Uso conforme al plan.
- $CPI < 1$ → Consumo superior al estimado.

Informes de Control Presupuestal

Los reportes presupuestales serán revisados al finalizar cada Sprint considerando:

- Horas invertidas.
- Horas estimadas.
- Trabajo completado.
- Consumo de reservas.
- Riesgos ocurridos.
- Desviaciones detectadas.

Los resultados permitirán actualizar futuras estimaciones y mejorar la precisión de planeación del proyecto.

Acciones Correctivas Presupuestales

Cuando se detecten desviaciones significativas podrán implementarse acciones como:

- Ajuste de estimaciones.
- Redistribución de actividades.
- Priorización de funcionalidades críticas.
- Reducción de retrabajo.
- Optimización de tiempos de integración.
- Replanificación de actividades futuras.

Las acciones correctivas serán evaluadas durante las reuniones de seguimiento y retrospectivas del proyecto.

6.4 Control de presupuesto

Objetivo del Control Presupuestal

El control presupuestal tiene como propósito supervisar el uso de recursos y el esfuerzo invertido durante el desarrollo del proyecto, permitiendo comparar el trabajo ejecutado respecto a la planeación inicial y detectar desviaciones relacionadas con tiempo, esfuerzo técnico o recursos operativos.

Debido a la naturaleza académica del proyecto, el control presupuestal se encuentra enfocado principalmente en el seguimiento de horas-hombre y consumo de recursos utilizados durante los Sprints.

Línea Base del Presupuesto

La línea base presupuestal estará conformada por:

- Estimación total de horas-hombre.
- Distribución de esfuerzo por actividades.
- Recursos asignados.
- Cronograma aprobado.
- Herramientas e infraestructura consideradas para el proyecto.

El esfuerzo total estimado corresponde a:

Toda modificación relacionada con recursos, tiempo o esfuerzo deberá pasar por el proceso formal de control de cambios.

Métricas de Costo y Esfuerzo

El seguimiento presupuestal será realizado principalmente mediante comparación entre:

- Horas estimadas vs horas reales.
- Trabajo planeado vs trabajo completado.
- Story Points comprometidos vs completados.
- Recursos utilizados vs recursos planificados.

Estas métricas permitirán evaluar el desempeño real del proyecto y detectar posibles desviaciones de esfuerzo.

Técnica de Valor Ganado

Para supervisar el desempeño del proyecto se utilizarán conceptos básicos de Valor Ganado (Earned Value Management — EVM).

Actual Cost (AC)

Representa el esfuerzo real invertido durante el desarrollo del proyecto.

Earned Value (EV)

Representa el valor del trabajo realmente completado respecto a la planeación inicial.

Cost Variance (CV)

Permitirá medir desviaciones presupuestales mediante:

$$CV=EV-AC$$

Interpretación:

- $CV > 0$ → Menor esfuerzo al estimado.
- $CV = 0$ → Esfuerzo conforme al plan.
- $CV < 0$ → Sobrecarga de esfuerzo.

Cost Performance Index (CPI)

Permitirá medir la eficiencia en el uso de recursos:

$$CPI=EV/AC$$

Interpretación:

- $CPI > 1$ → Uso eficiente de recursos.
- $CPI = 1$ → Uso conforme a la planeación.
- $CPI < 1$ → Consumo superior al estimado.

Aprobación de Gastos

El proyecto fue planeado considerando un costo monetario mínimo debido al uso de herramientas gratuitas, recursos académicos y equipos personales.

Sin embargo, en caso de requerirse gastos adicionales relacionados con:

- Pruebas.
- Infraestructura.
- Dispositivos.
- Transporte.
- Material de apoyo.

Estos deberán ser evaluados y aprobados previamente.

Las responsabilidades de aprobación serán:

Tipo de gasto	Responsable de aprobación
Gastos operativos menores	Scrum Master
Recursos técnicos o infraestructura	Product Owner
Gastos relacionados con operación del restaurante	Responsable del restaurante

En caso de aprobarse gastos extraordinarios, estos serán cubiertos de manera proporcional entre los integrantes del equipo.

Gestión de Reservas de Contingencia

Aunque el proyecto no contempla un presupuesto financiero formal, se considerará una reserva operativa de contingencia destinada a absorber desviaciones menores relacionadas con:

- Problemas de configuración.
- Fallas de integración.
- Repetición de pruebas.
- Corrección de errores críticos.
- Retrasos técnicos.
- Ajustes de infraestructura.

La reserva será administrada principalmente mediante:

- Redistribución de tiempo.
- Ajustes de carga de trabajo.
- Reprogramación de actividades.
- Uso adicional de horas de integración cuando sea necesario.

Frecuencia de Evaluación Presupuestal

El seguimiento presupuestal será realizado:

Actividad	Frecuencia
Revisión de horas reales	Semanal
Comparación esfuerzo estimado vs real	Al finalizar cada Sprint
Evaluación de métricas CPI y CV	Al finalizar cada Sprint
Revisión de consumo de recursos	Continua
Revisión de contingencias	Según incidencias

Acciones Correctivas

Cuando se detecten desviaciones importantes respecto a la planeación presupuestal podrán aplicarse acciones como:

- Redistribución de tareas.
- Ajuste de estimaciones.
- Repriorización de funcionalidades.
- Optimización de actividades.
- Replanificación de Sprints.
- Reducción de retrabajo.

Las acciones correctivas serán revisadas durante Sprint Reviews y Sprint Retrospectives.

Evidencia y Reportes

Toda la información relacionada con control presupuestal será documentada mediante:

- Reportes de Sprint.
- Historial de horas-hombre.
- Métricas de desempeño.
- Evidencias de Trello.
- Registros de incidencias.
- Reportes de retrospectivas.

La documentación permitirá mantener trazabilidad histórica sobre el desempeño y utilización de recursos durante el proyecto.

6.5 Aseguramiento de la calidad

Objetivo del Aseguramiento de la Calidad

El aseguramiento de la calidad tiene como propósito garantizar que tanto el proceso de desarrollo como el sistema de comanda digital cumplan con los requisitos funcionales, técnicos y de calidad establecidos para el proyecto.

Las actividades de calidad estarán orientadas a verificar el cumplimiento de:

- Requisitos definidos en la SRS.
- Historias de Usuario aprobadas.
- Estándares de desarrollo establecidos por el equipo.
- Procedimientos de Scrum.
- Criterios de aceptación del sistema.
- Buenas prácticas de ingeniería de software.

El proceso de calidad se encuentra alineado con las prácticas establecidas en:

- IEEE 12207.
- IEEE 15288.
- IEEE 16326.

Alcance del Aseguramiento de Calidad

Las actividades de calidad abarcarán:

- Procesos de desarrollo.
- Documentación técnica.
- Gestión de configuración.
- Integración de módulos.
- Validación funcional.
- Seguimiento de incidencias.
- Cumplimiento de estándares.
- Control de cambios.
- Validación operativa del sistema.

El aseguramiento de calidad será aplicado durante todo el ciclo de vida del proyecto.

Auditorías y Revisiones Internas

Se realizarán auditorías internas periódicas con el propósito de verificar el cumplimiento de los procesos y estándares definidos para el proyecto.

Las auditorías incluirán:

- Verificación de estándares de codificación.
- Revisión de cumplimiento de Scrum.
- Seguimiento del uso del tablero Kanban.
- Validación de documentación técnica.
- Revisión de evidencias y entregables.
- Verificación de trazabilidad de requisitos.
- Validación de control de versiones.
- Revisión de registros de incidencias.

Las revisiones permitirán detectar desviaciones y aplicar acciones correctivas de forma temprana.

Validación de Requisitos y Funcionalidades

Se verificará que las funcionalidades implementadas correspondan correctamente a:

- Historias de Usuario aprobadas.
- Requisitos funcionales de la SRS.
- Criterios de aceptación definidos.
- Objetivos operativos del restaurante.

La validación permitirá mantener consistencia entre los requerimientos iniciales y los módulos desarrollados.

Revisiones Técnicas

Las revisiones técnicas serán realizadas antes de integrar cambios a la rama principal del proyecto.

Las actividades de revisión incluirán:

- Revisión de código entre pares.
- Verificación de estándares de programación.
- Validación de integración.
- Revisión de estructura MVC.
- Validación de dependencias y configuraciones.
- Revisión de seguridad básica y control de accesos.

La revisión técnica permitirá reducir errores de integración y mejorar la mantenibilidad del sistema.

Estrategia de Pruebas

El proceso de validación incluirá diferentes niveles de pruebas orientadas a garantizar el correcto funcionamiento del sistema.

Pruebas Funcionales

Verificarán que cada módulo cumpla correctamente con las funcionalidades definidas.

Pruebas de Integración

Validarán la comunicación y sincronización entre:

- Cocina.
- Caja.
- Meseros.
- Administración.
- Gestión de mesas.
- Base de datos.

Pruebas de Caja Negra

Evaluarán el comportamiento del sistema desde la perspectiva del usuario sin considerar la lógica interna de implementación.

Pruebas Operativas

Simularán el flujo real del restaurante utilizando:

- Tablets Android.
- Laptops con Windows.
- Red Wi-Fi local.
- Dispositivos móviles del equipo.

Estas pruebas permitirán validar el comportamiento operativo del sistema bajo condiciones reales de uso.

Estándares Aplicados

El proyecto tomará como referencia los siguientes estándares:

Estándar	Aplicación
----------	------------

IEEE 12207	Procesos del ciclo de vida del software
IEEE 15288	Procesos de ingeniería de sistemas
IEEE 16326	Planeación y gestión del proyecto

La aplicación de estos estándares permitirá mantener organización, trazabilidad y control sobre el desarrollo del sistema.

Mejora Continua (PDCA)

El proyecto aplicará el ciclo de mejora continua PDCA (Plan — Do — Check — Act) para fortalecer los procesos de desarrollo y calidad.

Plan (Planificar)

- Definición de auditorías.
- Planeación de pruebas.
- Definición de métricas.
- Organización de revisiones.

Do (Ejecutar)

- Desarrollo de funcionalidades.
- Ejecución de pruebas.
- Aplicación de revisiones técnicas.
- Validación de entregables.

Check (Verificar)

- Evaluación de resultados.
- Análisis de métricas.
- Identificación de defectos.
- Comparación contra objetivos definidos.

Act (Actuar)

- Aplicación de acciones correctivas.
- Mejora de procesos.
- Ajuste de estimaciones.
- Optimización de integración y validación.

Las mejoras identificadas serán incorporadas progresivamente durante los siguientes Sprints.

Métricas de Calidad

Para evaluar el desempeño y calidad del proyecto se utilizarán las siguientes métricas:

Densidad de Defectos

Permitirá medir la cantidad de errores detectados respecto al tamaño o número de funcionalidades implementadas.

Número de Incidencias Críticas por Sprint

Permitirá identificar estabilidad y nivel de problemas críticos detectados durante cada iteración.

Sprint Velocity

Permitirá evaluar la capacidad real del equipo y detectar posibles afectaciones relacionadas con calidad o retrabajo.

Resultados de Pruebas Funcionales e Integración

Permitirá medir el porcentaje de funcionalidades aprobadas respecto al total de pruebas ejecutadas.

Bug Rate

Permitirá monitorear la frecuencia de errores detectados durante el desarrollo y validación del sistema.

Gestión de Incidencias de Calidad

Todas las incidencias detectadas durante auditorías o pruebas serán registradas considerando:

- Descripción del problema.
- Nivel de severidad.
- Módulo afectado.
- Responsable asignado.
- Estado de resolución.
- Evidencia relacionada.

Las incidencias serán monitoreadas hasta su corrección y validación final.

Evidencia y Trazabilidad de Calidad

Las actividades de calidad quedarán respaldadas mediante:

- Reportes de pruebas.
- Evidencias funcionales.
- Historial de incidencias.
- Resultados de auditorías.
- Registros de revisiones técnicas.
- Evidencias de integración.
- Actualizaciones en Trello y GitHub.

La documentación permitirá conservar trazabilidad histórica del aseguramiento de calidad durante todo el proyecto.

Responsabilidades de Calidad

Rol	Responsabilidad
Scrum Master	Supervisión de procesos y seguimiento
Product Owner	Validación funcional y aceptación
Equipo de desarrollo	Revisión técnica y corrección de incidencias
Responsable de pruebas	Validación funcional e integración
Todo el equipo	Cumplimiento de estándares y buenas prácticas

La participación colaborativa permitirá mantener control continuo sobre la calidad del sistema y del proceso de desarrollo.

6.6 Subcontratistas

Objetivo

La gestión de subcontratistas tiene como propósito definir los criterios relacionados con la posible participación de terceros en actividades técnicas, operativas o de soporte asociadas al proyecto.

Actualmente, el sistema será desarrollado completamente por el equipo interno del proyecto, por lo que no se contempla la contratación formal de proveedores externos durante la fase académica y de desarrollo inicial.

Situación Actual del Proyecto

Todas las actividades relacionadas con:

- Análisis.
- Diseño.

- Desarrollo.
- Integración.
- Validación.
- Documentación.
- Despliegue.
- Soporte técnico inicial.

Serán realizadas directamente por los integrantes del equipo del proyecto.

Debido a que no existen terceros involucrados en actividades críticas del sistema, actualmente no se requiere un proceso formal de selección, evaluación o control de subcontratistas.

Posibles Servicios Externos Futuros

En futuras versiones del sistema o durante etapas de operación real, podrían incorporarse servicios externos especializados relacionados con:

- Hosting o infraestructura en la nube.
- Soporte técnico especializado.
- Redes y conectividad.
- Impresión de tickets.
- Mantenimiento de hardware.
- Seguridad informática.
- Servicios de respaldo y almacenamiento.
- Soporte de dispositivos móviles o tablets.

La incorporación de servicios externos dependerá de:

- Necesidades operativas del restaurante.
- Escalabilidad del sistema.
- Recursos financieros disponibles.
- Complejidad técnica futura.

Criterios de Selección de Subcontratistas

En caso de requerirse subcontratación futura, los proveedores deberán evaluarse considerando:

- Compatibilidad técnica.
- Experiencia comprobable.
- Costos.
- Disponibilidad de soporte.
- Confiabilidad del servicio.
- Capacidad de integración.
- Seguridad y estabilidad operativa.

Supervisión y Control

Cuando existan servicios externos, el equipo del proyecto deberá supervisar:

- Cumplimiento de requisitos técnicos.
- Disponibilidad del servicio.
- Integración con el sistema.
- Cumplimiento de tiempos acordados.
- Riesgos asociados a dependencias externas.

Las actividades de supervisión serán coordinadas principalmente por:

Rol	Responsabilidad
Product Owner	Validación operativa y aprobación
Scrum Master	Seguimiento y coordinación
Equipo técnico	Validación de integración y compatibilidad

Riesgos Relacionados con Subcontratación

En futuras integraciones externas podrían existir riesgos relacionados con:

- Dependencia de servicios de terceros.
- Costos no previstos.
- Problemas de compatibilidad.
- Fallas de disponibilidad.
- Retrasos de soporte.
- Riesgos de seguridad o conectividad.

Estos riesgos deberán evaluarse antes de aprobar cualquier integración externa.

Evidencia y Contratos

En caso de contratar servicios externos en futuras etapas del proyecto, deberá mantenerse evidencia relacionada con:

- Acuerdos de servicio.
- Costos asociados.
- Configuraciones técnicas.
- Responsables asignados.
- Incidencias relacionadas.
- Validaciones de funcionamiento.

La documentación permitirá mantener control y trazabilidad sobre los servicios subcontratados.

6.7 Cierre del proyecto

Objetivo del Cierre del Proyecto

El cierre del proyecto tiene como propósito formalizar la finalización de las actividades de desarrollo, validar el cumplimiento de los objetivos establecidos y garantizar la entrega correcta del sistema de comanda digital junto con su documentación y evidencias correspondientes.

El proceso de cierre permitirá verificar que el proyecto cumple con los criterios de aceptación definidos, así como registrar los resultados obtenidos, lecciones aprendidas y desempeño general del proyecto respecto a la planificación inicial.

Criterios de Cierre

Para que el proyecto pueda considerarse formalmente finalizado deberán cumplirse los siguientes criterios:

- Cumplimiento del 100% de las Historias de Usuario aprobadas dentro de la línea base de requisitos.
- Finalización de las funcionalidades definidas en el Product Backlog aprobado.
- Todas las tareas del tablero Kanban deberán encontrarse en estado “Done” conforme a la Definition of Done (DoD).
- Finalización satisfactoria de:
 - Pruebas funcionales.
 - Pruebas de integración.
 - Pruebas de caja negra.
 - Validaciones operativas.
- Resolución de errores críticos identificados durante pruebas.
- Ejecución satisfactoria de pruebas del sistema dentro del entorno real del restaurante.
- Validación y aceptación funcional por parte del dueño o administrador del restaurante.
- Entrega completa de:
 - Documentación técnica.
 - Documentación de usuario.
 - Evidencias de pruebas.
 - Reportes del proyecto.
- Respaldo final del código fuente y documentación mediante GitHub y almacenamiento en nube.
- Cierre de incidencias críticas pendientes.
- Validación final de integración entre módulos.

El cumplimiento de estos criterios permitirá garantizar que el sistema se encuentra funcionalmente estable y listo para su entrega final.

Actividades de Cierre

Validación Final del Sistema

Se realizará una validación integral del sistema para comprobar:

- Correcto funcionamiento de módulos.
- Integración entre componentes.
- Flujo operativo completo.
- Sincronización de pedidos.
- Funcionamiento de autenticación.
- Estabilidad del entorno.

Las validaciones se efectuarán utilizando el entorno operativo definido para el restaurante.

Consolidación de Evidencias

Antes del cierre se recopilarán y organizarán:

- Evidencias funcionales.
- Resultados de pruebas.
- Reportes de incidencias.
- Métricas del proyecto.
- Historial de cambios.
- Evidencias de integración.
- Evidencias de Sprint Reviews y Retrospectives.

La consolidación permitirá mantener trazabilidad histórica del proyecto.

Respaldo y Liberación Final

El equipo realizará respaldos finales relacionados con:

- Código fuente.
- Base de datos.
- Configuraciones Docker.
- Documentación.
- Reportes.
- Evidencias del proyecto.

Los respaldos serán almacenados en:

- GitHub.
- Almacenamiento en nube.
- Equipos locales del proyecto.

Acta de Cierre

El acta de cierre será el documento formal mediante el cual el cliente o responsable del restaurante confirma que:

- El sistema cumple los criterios de aceptación establecidos.
- Las funcionalidades acordadas fueron entregadas.
- Las validaciones operativas fueron satisfactorias.
- El proyecto puede considerarse concluido.

El acta servirá como evidencia formal de aceptación y finalización del proyecto.

Informe Final del Proyecto

El informe final consolidará los principales resultados obtenidos durante el desarrollo del sistema.

El informe incluirá:

- Resumen general del proyecto.
- Funcionalidades implementadas.
- Resultados de pruebas.
- Métricas del proyecto.
- Comparación entre planeación y ejecución.
- Horas-hombre utilizadas.
- Riesgos ocurridos.
- Incidencias relevantes.
- Acciones correctivas aplicadas.
- Lecciones aprendidas.
- Resultados de calidad.
- Estado final del sistema.

El documento permitirá mantener registro histórico del desempeño general del proyecto.

Análisis de Desempeño vs Planeación

Se realizará una evaluación comparativa entre:

Elemento	Comparación
----------	-------------

Cronograma	Planeado vs real
Presupuesto	Línea base vs esfuerzo real
Hitos	Planeados vs cumplidos
Story Points	Estimados vs completados
Riesgos	Identificados vs ocurridos
Calidad	Objetivos vs resultados

El análisis permitirá identificar:

- Desviaciones relevantes.
- Áreas de mejora.
- Precisión de estimaciones.
- Efectividad del proceso Scrum.
- Cumplimiento de objetivos del proyecto.

Lecciones Aprendidas

Durante el cierre se documentarán las lecciones aprendidas relacionadas con:

- Planeación.
- Estimación.
- Integración.
- Validación.
- Trabajo colaborativo.
- Gestión de incidencias.
- Uso de herramientas.
- Comunicación del equipo.

Las lecciones aprendidas servirán como referencia para futuros proyectos y procesos de mejora continua.

Cierre Administrativo

El cierre administrativo incluirá:

- Verificación de documentación final.
- Organización de entregables.
- Cierre de tareas pendientes.
- Confirmación de respaldos.
- Archivo histórico del proyecto.
- Liberación de recursos utilizados.

Estas actividades permitirán finalizar formalmente el proyecto y conservar evidencia completa del trabajo realizado.

Responsables del Cierre

Rol	Responsabilidad
Scrum Master	Coordinación del cierre y seguimiento
Product Owner	Validación funcional y aceptación
Equipo de desarrollo	Corrección final y respaldo técnico
Responsable del restaurante	Validación operativa y conformidad

La participación conjunta permitirá garantizar un cierre formal y controlado del proyecto.

7. Entrega del producto

Objetivo de la Entrega del Proyecto

La entrega del proyecto tiene como propósito realizar la transferencia técnica, operativa y funcional del sistema de comanda digital al restaurante, garantizando que el sistema pueda operar correctamente dentro del entorno real de trabajo y que el personal cuente con la capacitación y documentación necesarias para su utilización. El proceso de entrega permitirá validar que el sistema cumple con los requisitos de aceptación establecidos y que se encuentra listo para su operación inicial.

Transferencia Técnica y Operativa

La transferencia técnica y operativa del sistema incluirá la puesta en funcionamiento del sistema dentro del restaurante y la validación de compatibilidad con la infraestructura disponible.

Las actividades de implementación contemplan:

- Instalación del sistema en equipos y tablets Android.
- Configuración del entorno Docker.
- Configuración de la base de datos.
- Configuración de usuarios y roles iniciales.
- Configuración de la red local Wi-Fi.
- Verificación de conectividad entre módulos.
- Validación de sincronización en tiempo real.
- Pruebas operativas en el entorno real del restaurante.

La transferencia se considerará completada cuando:

- El sistema funcione correctamente durante el periodo de validación.

- Las pruebas operativas sean satisfactorias.
- El responsable del restaurante apruebe el funcionamiento del sistema.

Estrategia de Entrega

La implementación del sistema se realizará de manera gradual con el propósito de reducir riesgos operativos durante la transición del proceso tradicional al sistema digital.

Durante la etapa inicial de operación:

- El sistema digital coexistirá temporalmente con las comandas en papel.
- El personal podrá utilizar ambos métodos mientras se adapta al nuevo flujo operativo.
- El equipo de desarrollo realizará monitoreo y seguimiento continuo.
- Se documentarán errores, incidencias y ajustes menores detectados durante el uso real.

La estrategia de despliegue contempla las siguientes fases:

Fase	Actividad
1	Instalación y configuración del sistema
2	Validación técnica interna
3	Capacitación del personal
4	Prueba piloto en entorno real
5	Corrección de incidencias detectadas
6	Liberación operativa final

La implementación gradual permitirá disminuir interrupciones operativas y facilitar la adaptación del personal.

Entregables Finales

Al finalizar el proyecto se entregarán los siguientes elementos:

- Sistema de comanda digital completamente funcional.
- Base de datos configurada.
- Sistema instalado y configurado en tablets Android y equipos del restaurante.
- Código fuente del sistema.
- Configuración Docker y entorno de despliegue.
- Manual técnico del sistema.
- Manual de usuario para meseros.
- Manual de operación para caja.
- Manual administrativo.
- Documentación del proyecto:

- PMP.
- Product Backlog.
- Evidencias.
- Resultados de pruebas.
- Reportes de Sprint.
- Resultados de pruebas funcionales e integración.
- Evidencias de validación operativa.
- Acta de aceptación y cierre del proyecto.

Todos los entregables deberán encontrarse actualizados y respaldados antes de la liberación final del sistema.

Documentación del Proyecto

Con el propósito de facilitar el uso, mantenimiento y continuidad operativa del sistema, se desarrollará documentación técnica y operativa asociada al proyecto.

La documentación incluirá:

- Manual de usuario para meseros.
- Manual operativo para caja.
- Manual administrativo.
- Manual técnico de instalación y despliegue.
- Guía básica de recuperación ante fallos.
- Configuración de Docker.
- Guía de respaldo y restauración.
- Procedimientos básicos de mantenimiento.

La documentación permitirá conservar trazabilidad y facilitar futuras actividades de soporte o mejora del sistema.

Capacitación

Se realizará capacitación presencial dirigida al personal involucrado en la operación del sistema.

La capacitación estará enfocada en:

- Uso del menú digital.
- Gestión de pedidos.
- Asignación de mesas.
- Cobro y generación de tickets.
- Gestión de cocina.
- Gestión de pedidos entregados.
- Uso básico del sistema administrativo.

- Solución de problemas operativos básicos.

Las sesiones incluirán:

- Explicaciones prácticas.
- Simulación de flujos reales.
- Uso de tablets Android.
- Ejercicios operativos.
- Validación de comprensión del sistema.

La capacitación permitirá reducir errores operativos durante la adopción inicial del sistema.

Soporte Post-Entrega

Después de la entrega oficial del sistema se establecerá un periodo de soporte técnico inicial para atender incidencias relacionadas con la puesta en marcha.

El soporte incluirá:

- Corrección de errores críticos.
- Ajustes menores de configuración.
- Validación de funcionamiento.
- Asistencia técnica básica.
- Monitoreo inicial del sistema.
- Seguimiento de incidencias operativas.

Periodo de Garantía

Se establece un periodo de garantía de 4 meses posteriores a la entrega oficial del sistema.

Durante este periodo el equipo brindará soporte relacionado con:

- Fallos críticos del sistema.
- Problemas de integración.
- Errores operativos derivados del sistema entregado.
- Ajustes menores relacionados con configuración inicial.

La garantía no contemplará:

- Modificaciones mayores al alcance original.
- Nuevas funcionalidades.
- Cambios estructurales no definidos en los requisitos iniciales.
- Problemas derivados de mal uso o modificaciones externas no autorizadas.

Validación de la Entrega

La entrega final será validada mediante:

- Ejecución de pruebas operativas.
- Revisión de entregables.
- Validación funcional del sistema.
- Confirmación de instalación correcta.
- Verificación de documentación.
- Aprobación del responsable del restaurante.

La validación quedará respaldada mediante el acta formal de aceptación y cierre del proyecto.

Responsables de la Entrega

Rol	Responsabilidad
Scrum Master	Coordinación general de la entrega
Product Owner	Validación funcional y aceptación
Equipo de desarrollo	Instalación, configuración y soporte
Responsable del restaurante	Validación operativa y aprobación

La participación conjunta permitirá realizar una transición controlada hacia la operación del sistema en el entorno real del restaurante.

Planes de procesos de soporte

8.1 Supervisión y Entorno de Trabajo

Objetivo

El propósito de la supervisión y del entorno de trabajo es garantizar que el equipo pueda desarrollar el proyecto en condiciones organizadas, colaborativas y controladas, manteniendo seguimiento continuo sobre el avance, la comunicación y la resolución de problemas durante el ciclo de vida del sistema.

Supervisión del Progreso

El Scrum Master realizará seguimiento continuo del proyecto mediante:

- Daily Scrum.
- Revisión del tablero Kanban.
- Seguimiento de tareas críticas.
- Revisión de hitos y entregables.
- Validación del cumplimiento de Sprint Goals.

El avance será monitoreado utilizando métricas como:

- Sprint Velocity.
- Número de tareas completadas.
- Story Points terminados por Sprint.
- Burn-down Chart.
- Bug Rate.

Estas actividades permitirán detectar retrasos, impedimentos y riesgos operativos de manera temprana.

Ambiente de Trabajo

El equipo trabajará en modalidad híbrida (presencial y remota), manteniendo reglas organizativas para asegurar coordinación y trazabilidad del trabajo.

Las reglas generales incluyen:

- Commit obligatorio al finalizar cada jornada.
- Uso obligatorio de GitHub para control de versiones.
- Actualización continua del tablero Kanban.
- Daily Scrum de máximo 15 minutos.
- Revisión de código entre pares antes de integrar cambios.
- Uso compartido de Docker para mantener entornos homogéneos.

El entorno de trabajo busca facilitar:

- Colaboración.
- Integración continua.
- Comunicación técnica.
- Organización de actividades.

Seguridad y Salud Ocupacional

Con el propósito de evitar sobrecarga y agotamiento del equipo se establecerán medidas relacionadas con bienestar y organización del trabajo.

Las medidas incluyen:

- Distribución equilibrada de tareas.
- Horarios razonables de desarrollo.
- Pausas durante jornadas extensas.
- Seguimiento de carga de trabajo.
- Comunicación abierta sobre bloqueos o dificultades.

Estas prácticas permitirán mantener estabilidad operativa y reducir riesgos asociados a fatiga o retrasos.

Bienestar del Equipo

El proyecto fomentará:

- Colaboración técnica.
- Aprendizaje compartido.
- Resolución temprana de conflictos.
- Comunicación respetuosa.
- Apoyo mutuo entre integrantes.

El Scrum Master y el Product Owner facilitarán mecanismos de comunicación para mantener un ambiente de trabajo colaborativo y organizado.

Mecanismos de Escalamiento

Problemas Técnicos del Sistema

Si un impedimento técnico no puede resolverse dentro de un periodo de 48 horas después de haber sido identificado en el Daily Scrum:

- El Scrum Master escalará el problema.
- El Product Owner evaluará posibles ajustes de cronograma.
- Se analizará necesidad de soporte externo o replanificación.

Problemas de Infraestructura o Equipo

En caso de fallas relacionadas con:

- Hardware.
- Conectividad.
- Configuración.
- Herramientas de desarrollo.

Se buscará resolver el problema en un máximo de 48 horas. Si el problema persiste:

- Se utilizará un equipo alternativo.
- Se restaurará el entorno mediante Docker y respaldos.

Problemas de Tiempo

Las actividades fueron estimadas considerando márgenes de contingencia para reducir riesgos relacionados con:

- Retrasos técnicos.
- Retrabajo.
- Problemas de integración.

- Incidencias críticas.

En caso necesario podrán ajustarse:

- Prioridades.
- Distribución de tareas.
- Alcance de Sprint.

8.2 Gestión de Decisiones

Objetivo

La gestión de decisiones tiene como propósito asegurar que las decisiones técnicas, operativas y organizacionales del proyecto sean documentadas, evaluadas y comunicadas correctamente.

Categorías de Decisión

Las decisiones podrán clasificarse como:

Categoría	Ejemplo
Técnica	Arquitectura, integración, base de datos
Operativa	Cronograma, asignación de tareas
Funcional	Cambios en requisitos
Calidad	Corrección de incidencias
Riesgos	Acciones de mitigación

Estrategia de Decisión

Las decisiones serán tomadas considerando:

- Impacto técnico.
- Riesgo.
- Tiempo disponible.
- Dependencias.
- Impacto operativo.
- Prioridad funcional.

Las decisiones críticas deberán ser discutidas durante:

- Sprint Planning.
- Sprint Review.
- Reuniones de seguimiento.

Registro de Decisiones

Todas las decisiones relevantes deberán quedar registradas mediante:

- Historial de cambios.
- Trello.
- GitHub.
- Documentación del PMP.
- Actas de reunión.

Riesgos por Sprint

Sprint	Riesgo	Mitigación
Sprint 1	Problemas de Docker y XAMPP	Validación de instalación y pruebas
Sprint 2	Mesas duplicadas y problemas de roles	Validaciones y controles de concurrencia
Sprint 3	Inconsistencias de stock y sincronización	Transacciones y pruebas integradas
Sprint 4	Fallos en el carrito de comparas y errores de actualización en tiempo real (polling).	Corrección de eventos JavaScript.
Sprint 5	Vulnerabilidades de seguridad y problemas de sesiones.	Implementación de regeneración de sesión y pruebas de seguridad.
Sprint 6	Bajo rendimiento del sistema y errores de integración final antes de producción.	Optimización de consultas, pruebas de rendimiento y monitoreo con Health Check.

8.3 Gestión de Riesgos

Objetivo

La gestión de riesgos busca identificar, analizar y mitigar incertidumbres que puedan afectar el cumplimiento del proyecto.

Estrategia de Identificación

Los riesgos serán identificados mediante:

- Daily Scrum.
- Retrospectivas.
- Revisiones técnicas.
- Resultados de pruebas.
- Seguimiento de incidencias.

Análisis y Priorización

Los riesgos serán evaluados considerando:

- Impacto.
- Probabilidad.
- Severidad.
- Afectación operativa.

Seguimiento

Los riesgos serán monitoreados continuamente durante:

- Sprint Reviews.
- Reuniones de seguimiento.
- Actividades de QA.

Matriz de Riesgos

Riesgo	Impacto	Probabilidad	Mitigación
Concurrencia de mesas	Alto	Medio	Bloqueo de registros en BD
Pérdida de sesión	Medio	Baja	Cookies persistentes
Inconsistencia de stock	Alto	Medio	Transacciones SQL
Caída del servidor local	Alto	Medio	Backups automáticos
Incompatibilidad de tablets	Medio	Medio	Diseño responsivo y pruebas en múltiples dispositivos.
Fallos en el carrito de compras	Alto	Alto	Validación de eventos JavaScript y pruebas funcionales continuas.
Error en actualización en tiempo real de cocina	Alto	Medio	Corrección de polling, endpoints y configuración CORS.
Errores de autenticación o roles.	Alto	Medio	Validaciones de acceso y pruebas de seguridad.

Lentitud del sistema bajo carga.	Alto	Medio	Optimización de consultas y pruebas de rendimiento.
Errores de integración entre módulos.	Alto	Medio	Pruebas de integración y validación.
Vulnerabilidades de seguridad no detectadas.	Alto	Baja	Pruebas de seguridad y revisiones periódicas.
Fallos de pruebas automatizadas.	Medio	Medio	Corrección de configuración PHPUnit y revisión de cobertura.

8.4 Gestión de Configuración

Control de Versiones

Git y GitHub serán utilizados para:

- Control de versiones.
- Gestión de ramas.
- Integración de cambios.
- Respaldo del código fuente.

Gestión de Cambios

Todo cambio deberá:

- Ser documentado.
- Ser revisado antes de integrarse.
- Mantener trazabilidad.
- Contar con validación funcional.

Estrategia de Versionamiento

El proyecto utilizará:

- Rama principal estable.
- Ramas de desarrollo por funcionalidad.
- Commits descriptivos.
- Integración controlada mediante revisión de código.

8.5 Gestión de la Información

Gestión Documental

Toda la documentación será almacenada y respaldada mediante:

- GitHub.
- OneDrive.
- Equipos locales.

Seguridad de la Información

Las medidas básicas de seguridad incluyen:

- Control de acceso por roles.
- Respaldo periódico.
- Protección de credenciales.
- Restricción de acceso administrativo.

Comunicación del Proyecto

La comunicación será realizada mediante:

- Daily Scrum.
- Reuniones quincenales.
- Trello.
- GitHub.
- Mensajería del equipo.

Mecanismo de Transferencia de Información

Durante reuniones periódicas se presentará:

- Evidencia funcional.
- Avance de Sprint.
- Cambios aprobados.
- Riesgos detectados.

Las decisiones deberán quedar documentadas dentro del historial del proyecto.

Matriz de Comunicación

Información a comunicar	Frecuencia	Formato/Herramienta	Emisor (Responsable)	Receptor (Stakeholders)
Estado del flujo de trabajo	Tiempo real	Tablero Kanban	Equipo de desarrollo	Scrum Master

Bloqueos o impedimentos	Diario	Daily Scrum	Desarrolladores	Scrum Master
Demo de incremento funcional	Quincenal	Reunión Presencial	Scrum Master	Product Owner / Dueño
Actualización del PMP y riesgos	Quincenal	Documento compartido	Scrum Master / Product Owner	Dueño
Test	Quincenal	Reunión presencial	Equipo de desarrollo	Scrum Master / Product Owner
Sprint Planning	Quincenal	Reunión presencial	Scrum Master / Product Owner	Equipo de desarrollo
Sprint Review	Diario	Tablero Kanban	Scrum Master / Product Owner	Scrum Master / Product Owner
Sprint Retrospective	Quincenal	Reunión presencial	Scrum Master / Product Owner	Scrum Master / Product Owner / Equipo de desarrollo

8.6 Aseguramiento de Calidad (QA)

Revisiones

Las revisiones incluirán:

- Revisión de código.
- Validación funcional.
- Revisión de integración.
- Verificación documental.

Estándares Aplicados

- IEEE 12207.
- IEEE 15288.
- IEEE 16326.
- Buenas prácticas Scrum.

8.7 Medición

Objetivo

Las métricas permitirán evaluar:

- Desempeño del equipo.
- Calidad.
- Eficiencia del desarrollo.
- Cumplimiento del cronograma.

Métricas Utilizadas

- Sprint Velocity.
- Burn-down Chart.
- Bug Rate.
- Throughput. (se descarto ya que es parecido a velocity ya que ambas miden produccion del equipo)
- Cycle Time.
- Lead Time. (Se descarta ya que termina siendo redundante con cicle time, burn-down y velocity)

Sprint Velocity

Se adjunta evidencia en la sección 9. Anexos

Burn-down Chart

Se adjunta evidencia en la sección 9. Anexos

Bug Rate

Se adjunta evidencia en la sección 9. Anexos

Cycle Time

Se adjunta evidencia en la sección 9. Anexos

8.8 Revisiones y Auditorías

Revisiones Internas

Se realizarán revisiones relacionadas con:

- Código fuente.
- Integración.
- Documentación.

- Cumplimiento de estándares.

Revisiones con Cliente

El Product Owner y el responsable del restaurante validarán:

- Funcionalidades.
- Flujo operativo.
- Requisitos cumplidos.
- Avance del proyecto.

8.9 Verificación y Validación

Pruebas Unitarias

Validación de funciones individuales del sistema.

Pruebas de Integración

Validación de comunicación entre módulos.

Pruebas de Estrés

Simulación de múltiples pedidos simultáneos para validar estabilidad del sistema.

Criterios de Aceptación

Cada Historia de Usuario deberá cumplir criterios funcionales antes de marcarse como terminada dentro del Sprint correspondiente.

8. Planes adicionales

Objetivo

Los planes adicionales tienen como propósito complementar la gestión del proyecto mediante estrategias específicas relacionadas con continuidad operativa, soporte técnico, seguridad, mantenimiento, mejora continua y administración posterior a la entrega del sistema.

Estos planes permiten fortalecer la estabilidad del proyecto y asegurar que el sistema pueda mantenerse operativo y administrable durante futuras etapas de uso y evolución.

8.1 Plan de Continuidad Operativa

Objetivo

Garantizar que el sistema pueda continuar operando ante fallos técnicos, incidencias menores o interrupciones parciales de infraestructura.

Estrategias de Continuidad

Se implementarán las siguientes medidas:

- Uso de respaldos periódicos del sistema.
- Control de versiones mediante GitHub.
- Replicación del entorno mediante Docker.
- Respaldo de documentación en nube.
- Recuperación rápida del entorno de desarrollo.

Recuperación ante Fallos

En caso de incidentes críticos se podrán realizar:

- Restauración de base de datos.
- Reconfiguración de contenedores Docker.
- Recuperación desde respaldos.
- Reinstalación controlada del sistema.

8.2 Plan de Mantenimiento

Objetivo

Definir las actividades necesarias para conservar la estabilidad y funcionamiento del sistema después de su entrega.

Tipos de Mantenimiento

Tipo	Descripción
Correctivo	Corrección de errores detectados
Preventivo	Validaciones periódicas y respaldos
Adaptativo	Ajustes por cambios operativos
Perfectivo	Mejoras menores y optimización

Actividades de Mantenimiento

- Actualización de configuraciones.
- Corrección de incidencias.
- Optimización de consultas SQL.
- Revisión de compatibilidad.
- Validación de respaldos.

- Revisión de integridad del sistema.

8.3 Plan de Seguridad

Objetivo

Establecer medidas básicas de protección para la información, usuarios y componentes del sistema.

Medidas de Seguridad

El sistema implementará:

- Autenticación mediante roles.
- Restricción de acceso administrativo.
- Validación de sesiones.
- Respaldo periódico de información.
- Protección básica de credenciales.
- Control de permisos.

Seguridad Operativa

Se recomienda:

- Mantener respaldos actualizados.
- Evitar compartir credenciales.
- Restringir accesos administrativos.
- Verificar dispositivos conectados.

8.4 Plan de Respaldo y Recuperación

Objetivo

Garantizar la disponibilidad de información crítica mediante respaldos organizados y mecanismos de recuperación.

Estrategia de Respaldo

Los respaldos incluirán:

- Base de datos.
- Código fuente.
- Configuración Docker.
- Documentación.
- Evidencias del proyecto.

Frecuencia de Respaldo

Recurso	Frecuencia
Código fuente	Cada 4 horas
Base de datos	Cada 4 horas
Documentación	Semanal
Evidencias	Al finalizar cada Sprint

Ubicación de Respaldos

Los respaldos serán almacenados en:

- GitHub.
- OneDrive.
- Equipos locales.
- Dispositivos externos cuando sea posible.

8.5 Plan de Mejora Continua

Objetivo

Establecer mecanismos para identificar oportunidades de mejora durante el desarrollo y operación del sistema.

Estrategia de Mejora

Las mejoras serán identificadas mediante:

- Sprint Retrospectives.
- Resultados de QA.
- Métricas ágiles.
- Incidencias detectadas.
- Retroalimentación del cliente.

Acciones de Mejora

Las acciones podrán incluir:

- Optimización de procesos.
- Ajustes de estimación.
- Mejoras de comunicación.
- Refinamiento del backlog.
- Automatización de tareas repetitivas.

Criterios de Éxito

Las mejoras serán evaluadas considerando:

- Reducción de incidencias.

- Mejora de Velocity.
- Reducción de tiempos de entrega.
- Estabilidad del sistema.
- Satisfacción del cliente.

8.6 Plan de Soporte Post-Entrega

Objetivo

Definir las actividades de soporte posteriores a la liberación oficial del sistema.

Alcance del Soporte

El soporte incluirá:

- Corrección de errores críticos.
- Ajustes menores.
- Asistencia técnica básica.
- Monitoreo inicial.
- Validación operativa.

Periodo de Soporte

Se establece un periodo de soporte y garantía de 4 meses posteriores a la entrega oficial del sistema.

8.7 Plan de Gestión del Conocimiento

Objetivo

Conservar y transferir conocimiento técnico y operativo relacionado con el sistema.

Estrategias

- Documentación técnica actualizada.
- Manuales de usuario.
- Evidencias de configuración.
- Historial de cambios.
- Registro de incidencias y soluciones.

Repositorio de Información

La información del proyecto será organizada dentro del repositorio utilizando una estructura documental centralizada:

/Documents_review/

La organización documental permitirá mantener:

- Trazabilidad.
- Control de configuración.
- Historial técnico.
- Evidencias del proyecto.

8.8 Plan de Gestión de Lecciones Aprendidas

Objetivo

Documentar experiencias obtenidas durante el proyecto para apoyar futuros desarrollos y procesos de mejora organizacional.

Registro de Lecciones Aprendidas

Las lecciones aprendidas serán recopiladas durante:

- Sprint Retrospectives.
- Cierre de Sprint.
- QA.
- Cierre del proyecto.

Información Registrada

Se documentarán:

- Problemas encontrados.
- Soluciones aplicadas.
- Riesgos ocurridos.
- Errores recurrentes.
- Buenas prácticas identificadas.
- Recomendaciones futuras.

Responsables

Plan	Responsable Principal
Continuidad operativa	Scrum Master
Seguridad y respaldos	Equipo técnico
Mejora continua	Scrum Master
Soporte post-entrega	Equipo de desarrollo
Gestión documental	Scrum Master y Product Owner

Anexos

Anexo A – Product Backlog

/Documentsreview/Documento final/PRODUCT_BACKLOGv1.1.pdf

Anexo B – Sprint Planning

/Documents_review/Sprint 2/Sprint Planning.pdf

/Documents_review/Sprint 3/Sprint Planning.pdf

/Documents_review/Sprint 4/Sprint Planning.pdf

/Documents_review/Sprint 5/Sprint Planning.pdf

/Documents_review/Sprint 6/Sprint Planning.pdf

/Documents_review/Sprint 7/Sprint Planning.pdf

Anexo C – Sprint Review

/Documents_review/Sprint 2/SPRINT REVIEW.pdf

/Documents_review/Sprint 3/ SPRINT REVIEW.pdf

/Documents_review/Sprint 4/ SPRINT REVIEW.pdf

/Documents_review/Sprint 5/ SPRINT REVIEW.pdf

/Documents_review/Sprint 6/ SPRINT REVIEW.pdf

/Documents_review/Sprint 7/ SPRINT REVIEW.pdf

Anexo E – Sprint Retrospective

/Documents_review/Sprint 2/Sprint Retrospective.pdf

/Documents_review/Sprint 3/ Sprint Retrospective.pdf

/Documents_review/Sprint 4/ Sprint Retrospective.pdf

/Documents_review/Sprint 5/ Sprint Retrospective.pdf

/Documents_review/Sprint 6/ Sprint Retrospective.pdf

/Documents_review/Sprint 7/ Sprint Retrospective.pdf

Anexo F – Daily Scrum

/Documents_review/Sprint 2/Daily Scrum.pdf

/Documents_review/Sprint 3/ Daily Scrum.pdf

/Documents_review/Sprint 4/ Daily Scrum.pdf

/Documents_review/Sprint 5/ Daily Scrum.pdf

/Documents_review/Sprint 6/ Daily Scrum.pdf

/Documents_review/Sprint 7/Daily Scrum.pdf

Anexo F –KPIs

A.1 Sprint Velocity

Métrica de Velocidad del Sprint (Sprint Velocity)

<i>Sprint</i>	<i>Story Point (SP) Comprometidos</i>	<i>Story Points (SP) Terminados</i>	<i>Eficiencia (Done/Planned)</i>	<i>Observaciones</i>
<i>Sprint 1</i>	7	7	1	Incentivar el trabajo en equipo
<i>Sprint 2</i>	32	32	1	Modificación de SP por observaciones del Product Owner
<i>Sprint 3</i>	18	13	0.72	En revisión
<i>Sprint 4</i>	24			
<i>Sprint 5</i>	18	66		
<i>Sprint 6</i>	11	51		
<i>Sprint 7</i>	29			
<i>Sprint 8</i>	47			

Por decisión del Scrum Master se generará en un archivo.xlsx para mayor practicidad su ubicacion es la siguiente:

/Documents_review/Sprint 2/Sprint Velocity.pdf

/Documents_review/Sprint 3/ Sprint Velocity.pdf

/Documents_review/Sprint 4/ Sprint Velocity.pdf

/Documents_review/Sprint 5/ Sprint Velocity.pdf
 /Documents_review/Sprint 6/ Sprint Velocity.pdf
 /Documents_review/Sprint 7/ Sprint Velocity.pdf

A.2 Burn-down Chart

Burn-down Chart: trabajo pendiente vs. el tiempo. Permite detectar si el equipo terminará a tiempo antes del cierre del Sprint.

13

<i>Sprint</i>	<i>Trabajo inicial (Total Backlog SP)</i>	<i>Trabajo Restante (Actual)</i>	<i>Trabajo restante (Ideal)</i>
<i>Sprint 1</i>	200	193	193
<i>Sprint 2</i>	193	161	161
<i>Sprint 3</i>	161	148	142
<i>Sprint 4</i>	224	197	197
<i>Sprint 5</i>	197	118	118
<i>Sprint 6</i>	118	67	67
<i>Sprint 7</i>	-	-	-
<i>Sprint 8</i>	-	-	-

Por decisión del Scrum Máster se generará en un archivo.xlsx para mayor practicidad su ubicacion es la siguiente:

/Documents_review/Sprint 2/Burn-Down.pdf
 /Documents_review/Sprint 3/ Burn-Down.pdf
 /Documents_review/Sprint 4/ Burn-Down.pdf
 /Documents_review/Sprint 5/ Burn-Down.pdf
 /Documents_review/Sprint 6/ Burn-Down.pdf
 /Documents_review/Sprint 7/ Burn-Down.pdf

A.3 Bug Rate

Calidad (Bug Rate): Número de errores reportados en las pruebas de aceptación sobre el total de HU entregadas.

<i>Sprint</i>	<i>Historia de Usuario (HU) Entregadas</i>	<i>Bugs Detectados (QA/UAT)</i>	<i>Tasa de Error (Bug HU)</i>
<i>BugSprint 1</i>	7	2	0.28
<i>Sprint 2</i>	8	3	0.37
<i>Sprint 3</i>	6	3	0.5
<i>Sprint 4</i>	8	3	-

<i>Sprint 5</i>	14	3	-
<i>Sprint 6</i>	9	0	0
<i>Sprint 7</i>	-	-	-
<i>Sprint 8</i>	-	-	-

Resultado: 5.3 historias por sprint

Por decisión del Scrum Master se generará en un archivo.xlsx para mayor practicidad su ubicacion es la siguiente:

/Documents_review/Sprint 2/Bug rate.pdf
 /Documents_review/Sprint 3/ Bug rate.pdf
 /Documents_review/Sprint 4/ Bug rate.pdf
 /Documents_review/Sprint 5/ Bug rate.pdf
 /Documents_review/Sprint 6/ Bug rate.pdf
 /Documents_review/Sprint 7/ Bug rate.pdf

A.4 Cycle Time

Version pasada de Cycle time

Cycle Time

HISTORIA	DÍAS
HU001	5
HU002	2
HU003	2
HU004	1
HU006	1
HU007	1
HU01	2
HU02	4
HU04	3
HU06	2
HU39	6
HU40	4
HU07	4
HU08	4
HU09	4
HU10	4
HU11	4

Promedio

$$P = (5+2+2+1+1+1+2+4+3+2+6+4+4+4+4+4+4)/17 = 3.17$$

Resultado

R = 3.17 días por historia

Revisiones internas

Revisiones con cliente

Por decisión conjunta del equipo de desarrollo, se optó por reorganizar las Historias de Usuario agrupándolas y ordenándolas según su categoría funcional, con el propósito de mejorar la organización del Product Backlog, facilitar la planificación de los Sprints y mantener una mayor trazabilidad entre los requisitos, módulos y actividades del proyecto.

su ubicación es la siguiente:

/Documents_review/Sprint 2/Cycle time.pdf
/Documents_review/Sprint 3/ Cycle time.pdf
/Documents_review/Sprint 4/ Cycle time.pdf
/Documents_review/Sprint 5/ Cycle time.pdf
/Documents_review/Sprint 6/ Cycle time.pdf
/Documents_review/Sprint 7/ Cycle time.pdf

A.5 Lead Time

Lead Time

Se creo el 30/2/2026

Historia	Días
HU001	6
HU002	3
HU003	3
HU004	2
HU005	3
HU006	2
HU007	2
HU01	2
HU02	7
HU04	11
HU06	2
HU39	6
HU40	14
HU07	18
HU08	18
HU09	18
HU10	En proceso

HU11	En proceso
------	------------

Promedio:

$$(6+3+3+2+3+2+2+2+7+11+2+6+14+18+18+18)/16 = 7.3$$

Resultado:

7.3 días por historia aproximadamente

A.6 Throughput

Throughput

SPRINT	HISTORIAS COMPLETADAS
SPRINT 1	7
SPRINT 2	6
SPRINT 3	3

Throughput promedio:

$$P = (7+6+3)/3 = 5.3$$

Anexo G – Kanban

/Documents_review/Documento final/Kanban.pdf

Anexo H – Cronograma de los sprints

/Documents_review/Documento final/Cronograma de Sprints.pdf

Anexo I – Presupuesto

/Documents_review/Documento final/Presupuesto.pdf

Anexo J – Plan de entrenamiento

/Documents_review/Documento final/Training-Plan.pdf

Anexo K – Gestión de Configuración e integración de cambios Git

/Documents_review/Documento final/Gestion de configuracion e Integración de Cambios Git.pdf

Anexo L – Pruebas

/Documents_review/Pruebas_Comanda/Casos_de_uso.pdf

/Documents_review/Pruebas_Comanda/Plan_de_pruebas.pdf

/Documents_review/Pruebas_Comanda/Casos_de_pruebas.pdf

/Documents_review/Pruebas_Comanda/Diseño_pruebas_caja_negra.pdf
/Documents_review/Pruebas_Comanda/Reporte_pruebas.pdf